

[ÜNİVERSİTE / OKUL ADI]
[FAKÜLTE / MESLEK YÜKSEKOKULU]
[BÖLÜM ADI]

YAZ STAJI RAPORU



DEPREMİN NABZI

Flutter ile Geliştirilmiş Çok Kaynaklı Deprem
İzleme ve Afet Hazırlık Uygulaması

*AFAD ve Kandilli açık verileri üzerinde konuma özel
hissedilirlik tahmini yapan mobil uygulama*

Hazırlayan	Tuna Kılıç
Öğrenci Numarası	
Sınıf / Dönem	
Staj Yapılan Kurum	
Kurum Adresi	
Staj Başlangıç Tarihi	
Staj Bitiş Tarihi	
Toplam Staj Süresi	30 iş günü
Kurum Yetkilisi	
Staj Danışmanı	

ÖZET

Bu rapor, 30 iş günü süren yaz stajı kapsamında baştan sona tasarlanıp geliştirilen “**Deprem Nabzı**” adlı çapraz platform mobil uygulamasını konu almaktadır. Uygulama, Türkiye ve çevresinde meydana gelen depremleri AFAD (Afet ve Acil Durum Yönetimi Başkanlığı) ve Boğaziçi Üniversitesi Kandilli Rasathanesi'nin açık verilerinden çekerek listeler, harita üzerinde gösterir ve kullanıcıya afet öncesi hazırlık desteği sunar.

Projenin diğer deprem uygulamalarından ayrıldığı temel nokta, ham büyüklük değerini göstermekle yetinmeyip kullanıcının kendi kaydettiği konumlarda **hissedilecek sarsıntı şiddetini tahmin etmesidir**. Bu tahmin, Allen, Wald ve Worden (2012) tarafından yayımlanan ve aktif kabuk bölgeleri için geliştirilmiş şiddet sönümlenme (intensity attenuation) modelinin hiposantr mesafeli sürümü ile yapılmaktadır. Böylece kullanıcı “4.1 büyüklüğünde deprem” gibi soyut bir bilgi yerine “*Evinizden 340 km uzakta — hissetmeyeceksiniz*” şeklinde doğrudan kendisini ilgilendiren bir cevap almaktadır.

Uygulama Flutter/Dart ile geliştirilmiş, yaklaşık **11.000 satır Dart kodu** ve **143 birim testi** içermektedir. Mimari; model, servis, durum (state) ve arayüz katmanlarına ayrılmış; durum yönetimi harici bir paket kullanılmadan Flutter'ın kendi `ChangeNotifier` altyapısıyla çözülmüştür. İki farklı veri kaynağının farklı JSON şemaları, farklı tarih biçimleri, farklı koordinat sıralamaları ve farklı gecikme davranışları tek bir ortak veri modeli altında birleştirilmiştir.

Proje boyunca öne çıkan mühendislik kararları; kaynak gecikmesine karşı geliştirilen otomatik kaynak seçim mekanizması, iki kaynaktan gelen aynı depremin zaman ve mesafe toleransı ile tekilleştirilmesi, acil durum modülündeki dört basamaklı konum yedekleme zinciri ve uzun periyotlu bildirimlerin yeniden planlamayla çözülmesidir. Ayrıca uygulama, teknik olarak yapamadığı şeyleri (deprem tahmini ve erken uyarı) gizlemek yerine kullanıcıya açıkça anlatan bir **dürüstlük ilkesi** üzerine kurulmuştur; bu, bir güvenlik ve afet uygulamasında kritik bir tasarım kararıdır.

Anahtar Kelimeler: Flutter, Dart, mobil uygulama geliştirme, deprem izleme, AFAD, Kandilli Rasathanesi, REST API, Mercalli şiddet ölçeği, sönümlenme modeli, haversine, OpenStreetMap, yerel bildirim, birim testi, afet hazırlığı.

İÇİNDEKİLER

ÖZET	2
1. GİRİŞ	5
1.1. Stajın Amacı ve Kapsamı	5
1.2. Problem Tanımı	5
1.3. Projenin Hedefleri	5
1.4. Raporun Yapısı	6
2. PROJENİN GENEL TANITIMI	7
2.1. Uygulama Kimliği	7
2.2. Hedef Kitle ve Kullanım Senaryoları	7
2.3. Temel Özellikler	7
2.4. Ekran Yapısı	8
2.5. Benzer Uygulamalardan Farkı	8
3. KULLANILAN TEKNOLOJİLER VE GELİŞTİRME ORTAMI	9
3.1. Flutter ve Dart	9
3.2. Teknoloji Seçim Gerekçeleri	9
3.3. Kullanılan Paketler	9
3.4. Geliştirme Ortamı ve Araçlar	10
3.5. Sürüm Kontrolü	10
4. SİSTEM MİMARİSİ	11
4.1. Katmanlı Mimari Yaklaşımı	11
4.2. Proje Klasör Yapısı	11
4.3. Veri Akışı	11
4.4. Durum Yönetimi	12
4.5. Filtrelerin İkiye Ayrılması	13
5. VERİ KAYNAKLARI VE API ENTEGRASYONU	14
5.1. AFAD Deprem Servisi	14
5.2. Kandilli Rasathanesi Verisi	14
5.3. İki Kaynağın Karşılaştırılması	15
5.4. Otomatik Kaynak Seçimi	15
5.5. Kayıt Tekilleştirme (Birleştirme)	16
5.6. Kapsama Şeffaflığı	16
5.7. Haber Beslemesi ve Harita Servisi	17
6. UYGULAMA MODÜLLERİ	18
6.1. Model Katmanı	18
6.2. Servis Katmanı	18
6.3. Yardımcı Sınıflar	19
6.4. Ekranlar ve Bileşenler	19
7. HİSSEDİLİRLİK TAHMİNİ MODÜLÜ	20
7.1. Problemin Tanımı	20
7.2. Kullanılan Bilimsel Model	20
7.3. Mesafe Hesabı: Haversine Formülü	20
7.4. Hiposantr Mesafesi	21
7.5. Şiddet Denklemi ve Katsayılar	21
7.6. Sayıdan Kategoriye Dönüşüm	21

7.7. Geçerlilik Sınırları ve Dürüstlük İlkesi	22
7.8. Örnek Hesap	22
8. BİLDİRİM VE HAZIRLIK MODÜLÜ	23
8.1. Üç Kavramın Ayrımı	23
8.2. Erken Uyarı Neden Yapılamıyor	23
8.3. Hatırlatma Planlama Algoritması	23
8.4. Bildirim Kimliği Çakışması	24
8.5. Paket Sürüm Seçimi	24
9. ACİL DURUM MODÜLÜ	25
9.1. Basamaklı Konum Yedeklemesi	25
9.2. İletişim Kanalları	25
9.3. Platform Kısıtları ve Şeffaflık	25
9.4. Yanlışlıkla Tetiklenmeye Karşı Koruma	26
9.5. Android Paket Görünürlüğü Sorunu	26
10. DUYURULAR MODÜLÜ	27
10.1. Neden Bülten Üretimi	27
10.2. Artçı Sayımı ve Eğilim	27
10.3. Haber Süzme Katmanları	27
10.4. Telif ve Etik	28
11. ARAYÜZ VE KULLANICI DENEYİMİ TASARIMI	29
11.1. Görsel Dil ve Tema	29
11.2. Bilgi Hiyerarşisi	29
11.3. Algılanan Performans	29
11.4. Erişilebilirlik	30
11.5. Uygulama Kimliği: İkon ve Açılış	30
12. TEST VE KALİTE GÜVENCESİ	31
12.1. Test Stratejisi	31
12.2. Test Kapsamı	31
12.3. Örnek Test Senaryoları	31
12.4. Statik Analiz	32
13. KARŞILAŞILAN SORUNLAR VE ÇÖZÜMLERİ	33
14. BİLİNER SINIRLAR	35
15. GELİŞTİRİLEBİLECEK YÖNLER	36
16. ÇALIŞMA TAKVİMİ (30 İŞ GÜNÜ)	37
17. KAZANIMLAR VE DEĞERLENDİRME	40
18. SONUÇ	42
KAYNAKÇA	43
EK A — Proje Dizin Yapısı	44
EK B — Örnek API Yanıtları	45
EK C — Kurulum ve Çalıştırma	47

1. GİRİŞ

1.1. Stajın Amacı ve Kapsamı

Yaz stajı, teorik derslerde edinilen bilginin gerçek bir yazılım ürünü üretme sürecinde sınındığı bir uygulama dönemidir. Bu staj kapsamında hedef, bir konuyu yüzeysel biçimde örneklemek değil; **fikir aşamasından test edilmiş, çalışan ve yayımlanabilir bir mobil uygulamaya kadar tüm yazılım geliştirme yaşam döngüsünü tek başına yürütmek** olarak belirlenmiştir.

30 iş günlük süre boyunca aşağıdaki aşamaların tamamı bizzat gerçekleştirilmiştir:

- Problem tanımı ve gereksinim analizi
- Veri kaynaklarının araştırılması, erişilebilirliğinin ve güvenilirliğinin ölçülmesi
- Mimari tasarım ve katman ayrımı
- Arayüz tasarımı ve kullanıcı deneyimi kararları
- Kodlama (Dart / Flutter)
- Birim testlerinin yazılması
- Hata ayıklama, performans iyileştirme ve platform uyum çalışmaları
- Belgelendirme ve sürüm kontrolü

Proje konusu olarak deprem izleme uygulamasının seçilmesi rastlantısal değildir. Türkiye, yüzölçümünün büyük bölümü aktif fay hatları üzerinde bulunan bir ülkedir ve deprem, toplumun gündeminden hiç düşmeyen bir konudur. Bu nedenle proje hem teknik açıdan zorlayıcı (gerçek zamanlı veri, harita, coğrafi hesaplama, bildirim, konum servisleri) hem de toplumsal karşılığı olan bir konu üzerinde çalışma imkânı sunmuştur.

1.2. Problem Tanımı

Mevcut deprem uygulamalarının hemen tamamı aynı işlevi yerine getirmektedir: resmî kurumlardan gelen deprem kayıtlarını bir listede sıralamak ve harita üzerinde işaretlemek. Bu yaklaşımın üç temel eksikliği vardır.

Birinci eksik: büyüklük tek başına anlamlı değildir. Kullanıcıya “Malatya Akçadağ’da 4.1 büyüklüğünde deprem” bilgisi verildiğinde, kullanıcının gerçekte merak ettiği soru cevapsız kalır. O soru “kaç şiddetinde?” değil, **“beni etkiler mi?”** sorusudur. 300 km uzaktaki 5.0 büyüklüğündeki bir deprem hiç hissedilmezken, 10 km yakındaki 4.0 büyüklüğündeki bir deprem insanı uykudan uyandırabilir. Büyüklük; mesafe ve derinlikten bağımsız olarak sunulduğunda kullanıcının kendi durumunu değerlendirmesine yardımcı olmaz.

İkinci eksik: uygulamalar yalnızca “ne oldu”yu anlatır. Deprem bilgisi doğası gereği geçmişe dönüktür; kullanıcı bildirimi aldığı anda olay çoktan bitmiştir. Buna karşılık deprem *olmadan önce* yapılabilecek çok şey vardır: deprem çantasının hazırlanması, ev içi güvenlik önlemlerinin alınması, aile buluşma noktasının belirlenmesi. Bu hazırlıkların ihmal edilme nedeni bilgisizlik değil, unutulmalarıdır. Uygulamaların büyük bölümü bu alana hiç girmez.

Üçüncü eksik: yanıltıcı vaatler. Bir kısım uygulama ve içerik “deprem tahmini” ya da “erken uyarı” iddiasında bulunmaktadır. Deprem tahmini bugün bilimsel olarak mümkün değildir; erken uyarı ise gerçektir ancak yalnızca sismik istasyon ağına ya da işletim sistemi düzeyinde erişime sahip sistemlerle sağlanabilir. Açık veri API’si üzerinden çalışan bir uygulamanın erken uyarı vermesi teknik olarak imkânsızdır, çünkü kullandığı veri zaten deprem gerçekleşip analiz edildikten sonra yayımlanmaktadır.

1.3. Projenin Hedefleri

Yukarıdaki üç eksik doğrudan proje hedeflerine dönüştürülmüştür:

No	Hedef	Karşılığı
H1	Kişiselleştirilmiş etki bilgisi	Kullanıcının kaydettiği konumlarda beklenen sarsıntı şiddetinin bilimsel bir sönümlenme modeliyle tahmin edilmesi
H2	Afet öncesi işlevsellik	Hazırlık kontrol listesi ve periyodik yerel bildirimlerle deprem olmadan önce değer üretmek
H3	Dürüst iletişim	Uygulamanın yapabildikleri kadar yapamadıklarının da açıkça anlatılması; tahmin ve erken uyarı konusunda yönlendirme
H4	Kesintisiz çalışma	İki bağımsız veri kaynağı ve otomatik geçiş ile tek bir servisin çökmesinde uygulamanın çalışmaya devam etmesi
H5	Gizlilik	Kayıtlı konumların yalnızca cihazda tutulması, hiçbir sunucuya veri gönderilmemesi
H6	Maliyetsiz altyapı	API anahtarı, faturalandırma hesabı veya arka uç sunucusu gerektirmeyen bir mimari
H7	Erişilebilirlik	Ekran okuyucu desteği, renk körlüğüne uygun kodlama, büyük yazı tipi uyumu

1.4. Raporun Yapısı

Raporun ilk üç bölümü projeyi ve kullanılan teknolojileri tanıtır. Dördüncü ve beşinci bölümler sistem mimarisini ve veri kaynaklarıyla entegrasyonu ele alır. Altı ile on birinci bölümler arası, uygulamanın modüllerini teknik ayrıntılarıyla açıklar; bu bölümlerde tasarım kararlarının gerekçelerine ve kod örneklerine yer verilir. On ikinci bölüm test ve kalite güvencesi çalışmalarını, on üç ile on beş arası bölümler karşılaşılan sorunları, mevcut sınırları ve gelecek geliştirme yönlerini kapsar. On altıncı bölüm 30 günlük çalışma takvimini, son iki bölüm ise kazanımların değerlendirmesini ve sonucu içerir.

2. PROJENİN GENEL TANITIMI

2.1. Uygulama Kimliği

Özellik	Değer
Uygulama adı	Deprem Nabzı
Paket adı (teknik)	deprem_takip
Kuruluş tanımlayıcısı	com.tunakilic
Sürüm	1.0.0+1
Platformlar	Android, iOS (ayrıca macOS ve Web derlenebilir)
Geliştirme çerçevesi	Flutter 3.27+ / Dart SDK 3.6+
Kod büyüklüğü	10.982 satır Dart (39 kaynak dosya, 7 test dosyası)
Test sayısı	143 birim testi, 33 test grubu
Dış bağımlılık	9 paket (tamamı açık kaynak, API anahtarı gerektirmez)
Arka uç sunucusu	Yok — tamamen istemci tarafında çalışır

2.2. Hedef Kitle ve Kullanım Senaryoları

Uygulamanın hedef kitlesi, deprem riski taşıyan bölgelerde yaşayan ya da yakınları böyle bölgelerde bulunan genel kullanıcılardır. Sismolog veya afet uzmanı gibi profesyonel kullanıcılar hedeflenmemiştir; bu nedenle arayüz teknik terim yoğunluğundan kaçınacak biçimde tasarlanmıştır.

Belirlenen üç ana kullanım senaryosu:

Senaryo	Kullanıcının Sorusu	Uygulamanın Cevabı
S1 — Sarsıntı sonrası	“Az önce salladı, ne oldu? Kaç büyüklüğündeydi, nerede?”	Liste sekmesinin en üstünde durum kartı; 60 saniyede bir otomatik yenilenen kayıt listesi
S2 — Uzaktan takip	“Ailem Malatya’da. Orada bir şey oldu mu, hissettiler mi?”	Yerlerim özelliği ile aile evinin kaydedilmesi ve her deprem için o noktadaki tahmini etki
S3 — Hazırlık	“Deprem çantamı en son ne zaman kontrol ettim?”	Hazırlık kontrol listesi ve periyodik yerel bildirimler

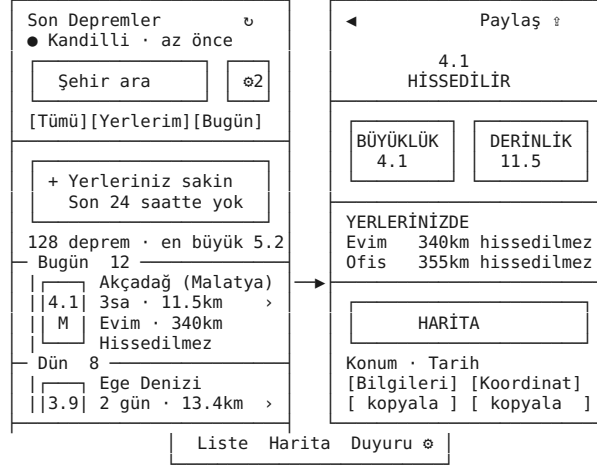
2.3. Temel Özellikler

Uygulama beş sekmeli bir yapıya sahiptir. Her sekmenin sağladığı işlevler aşağıda özetlenmiştir.

Sekme	İşlevler
Liste	Kişisel durum kartı, şehir araması, tek dokunuşluk hızlı filtreler (Tümü / Yerlerim / Son 1 saat / Bugün / Hissedilenler 3.0+ / Güçlü 4.5+), günlere ayrılmış yapışkan başlıklı liste, ayrıntılı filtre paneli, aşağı çekerek yenileme, iskelet yükleme animasyonu
Harita	OpenStreetMap tabanlı etkileşimli harita, büyüklüğe göre renk ve boyut kodlu işaretçiler, işaretçiye dokununca açılan alt özet kartı, detay ekranına geçiş
Duyurular	TRT Haber RSS beslemesinden süzölmüş deprem haberleri (görselleriyle birlikte), resmî veriden üretilen deprem bültenleri, artçı sayımı ve eğilimi, tahmin iddiası içeren haberler için uyarı etiketi
Acil	Basılı tutmayla tetiklenen acil durum butonu, konum içeren hazır mesaj, SMS ve WhatsApp kanalları, 112 kısayolu, “İyiyim” mesajı, acil kişi yönetimi
Ayarlar	Veri kaynağı seçimi (Otomatik / AFAD / Kandilli), yerlerim yönetimi, hazırlık kontrol listesi ve hatırlatma ayarları, erken uyarı bilgilendirme ekranı, hakkında bilgileri

2.4. Ekran Yapısı

Aşağıdaki şema, liste sekmesi ile detay ekranı arasındaki geçişi ve ekranlarda yer alan bilgi bloklarını göstermektedir.



Şekil 2.1 — Liste sekmesi (sol) ve deprem detay ekranı (sağ) şeması

2.5. Benzer Uygulamalardan Farkı

Piyasadaki deprem uygulamalarıyla karşılaştırıldığında projenin ayrıştığı noktalar şunlardır:

Konu	Yaygın Yaklaşım	Bu Projedeki Yaklaşım
Bilgi sunumu	Ham büyüklük değeri	Kullanıcının konumundaki tahmini hissedilirlik kategorisi
Zamanlama	Yalnızca deprem sonrası	Deprem öncesi hazırlık modülü ile sürekli değer üretimi
Veri kaynağı	Tek kaynak	İki kaynak + tazeliğe göre otomatik seçim
Sınırların iletimi	Belirtilmez veya küçük punto ile geçirilir	Ayrı bir bilgilendirme ekranı; her tahminde geçerlilik etiketi
Gizlilik	Konum sunucuya gönderilir	Konum yalnızca cihazda; takip için konum izni dahi istenmez
Maliyet	Ücretli harita API'si (Google Maps vb.)	OpenStreetMap — anahtarsız ve ücretsiz

3. KULLANILAN TEKNOLOJİLER VE GELİŞTİRME ORTAMI

3.1. Flutter ve Dart

Flutter, Google tarafından geliştirilen açık kaynaklı bir kullanıcı arayüzü çerçevesidir. Tek bir kod tabanından Android, iOS, web, Windows, macOS ve Linux hedeflerine derleme yapabilmesi temel özelliğidir. Flutter, platformun kendi arayüz bileşenlerini kullanmak yerine *Skia* (yeni sürümlerde *Impeller*) grafik motoru üzerinde kendi bileşenlerini doğrudan çizer. Bu yaklaşımın sonucu, aynı kodun her platformda piksel düzeyinde aynı görünmesidir.

Dart, Flutter'ın kullandığı programlama dilidir. Statik tipli, sınıf tabanlı ve null güvenliği (null safety) destekleyen bir dildir. Geliştirme sırasında JIT (Just-In-Time) derlemesi sayesinde *hot reload* özelliği sunar; kod değişikliği uygulamanın durumu kaybolmadan saniyeler içinde ekrana yansır. Yayın sürümünde ise AOT (Ahead-Of-Time) derlemesiyle native makine koduna çevrilir.

Flutter'da her arayüz ögesi bir *widget*'tır ve widget'lar iç içe geçerek bir ağaç oluşturur. Bu bildirimsel (declarative) yaklaşımda geliştirici “ekranı nasıl değiştireceğini” değil, “belirli bir durumda ekranın nasıl görünmesi gerektiğini” tanımlar; değişiklikleri çerçeve kendisi hesaplar.

3.2. Teknoloji Seçim Gerekçeleri

Karar	Alternatifler	Gerekçe
Flutter	React Native, Kotlin + Swift ayrı ayrı, Ionic	Tek kod tabanı ile iki platform; hot reload sayesinde kısa geri bildirim döngüsü; zengin ve hazır bileşen kütüphanesi; 30 günlük sürede iki ayrı native uygulama yazmak gerçekçi değildi
OpenStreetMap + flutter_map	Google Maps, Mapbox	API anahtarı ve faturalandırma hesabı gerektirmemesi; açık lisans; öğrenci projesi için maliyetsiz ve sürdürülebilir
ChangeNotifier	Provider, Riverpod, BLoC, GetX	Uygulamanın ölçeği için Flutter'ın yerleşik aracı yeterli; ek bağımlılık ve öğrenme maliyeti yok; durum yönetiminin nasıl çalıştığının kavranmasını sağladı
shared_preferences	SQLite (sqflite), Hive, Isar	Saklanan veri küçük ve tabloya ihtiyaç duymayan yapıda (tercihler, konum listesi, hazırlık durumu); JSON serileştirme ile yeterli
Sunucusuz mimari	Firebase, özel arka uç	Barındırma maliyeti sıfır; gizlilik açısından veri cihazdan çıkmıyor; bakım yükü yok
Clipboard tabanlı paylaşım	share_plus paketi	Paketin güncel sürümü Flutter 3.38+ istiyordu; ayrıca iPad'de <code>sharePositionOrigin</code> verilmediğinde çökme sorunu vardı

3.3. Kullanılan Paketler

Projede kullanılan dış bağımlılıklar ve seçim gerekçeleri aşağıdadır. Sürüm aralıkları bilinçli olarak geniş tutulmuş, yalnızca API kararlılığı riski taşıyan pakette üst sınır sabitlenmiştir.

Paket	Sürüm	Kullanım Amacı
http	^1.2.0	AFAD ve Kandilli API'lerine HTTP GET istekleri; RSS beslemelerinin indirilmesi
flutter_map	^8.0.0	OpenStreetMap döşemelerini gösteren etkileşimli harita bileşeni
latlong2	^0.9.1	Enlem/boylam veri tipi; flutter_map ile birlikte zorunlu
shared_preferences	^2.3.0	Kullanıcı tercihleri, kayıtlı konumlar, acil kişiler ve hazırlık durumunun cihazda kalıcı saklanması
flutter_local_notifications	^19.0.0	Hazırlık hatırlatmaları için zamanlanmış yerel bildirimler
timezone	^0.10.0	Zamanlanmış bildirimlerin doğru saat diliminde tetiklenmesi
geolocator	>=13.0.0 <15.0.0	Acil durum modülünde GPS konumunun okunması ve izin yönetimi
url_launcher	^6.3.0	SMS, WhatsApp, telefon araması ve tarayıcı bağlantılarının açılması
xml	>=6.3.0 <8.0.0	TRT Haber RSS beslemesinin (XML) ayrıştırılması
flutter_lints	^5.0.0	Geliştirme bağımlılığı — statik kod analizi kuralları

Sürüm aralığı stratejisi

Paketlerin çoğunda alt sınır geniş tutulmuştur; böylece `pub`, mevcut Flutter sürümüyle uyumlu en uygun sürümü kendisi seçebilir. Tek istisna `flutter_local_notifications`'tır: bu paket sürümleri arasında API'sini sık değiştirdiği için `^19.0.0` ile sabitlenmiştir. Ayrıntısı 8.5. bölümde.

3.4. Geliştirme Ortamı ve Araçlar

Bileşen	Ayrıntı
İşletim sistemi	macOS
SDK	Flutter 3.27+ (Dart 3.6+ içerir)
Kod düzenleyici	Visual Studio Code (Flutter ve Dart eklentileriyle), Android Studio
Android test ortamı	Android Studio Emülatörü
iOS test ortamı	Xcode iOS Simulator
Sürüm kontrolü	Git / GitHub
Yardımcı betikler	<code>basla.command</code> , <code>calistir-ios.command</code> , <code>kontrol.command</code>
Statik analiz	<code>analysis_options.yaml</code> + <code>flutter_lints</code>

Geliştirme sürecini hızlandırmak için üç yardımcı kabuk betiği yazılmıştır. `basla.command` bağımlılıkları indirip uygulamayı varsayılan cihazda başlatır; `calistir-ios.command` iOS Simulator'ı açıp uygulamayı oraya yükler; `kontrol.command` ise derleme öncesi statik analiz ve testleri tek komutta çalıştırır. Bu betikler, tekrarlanan komut dizilerinin hatasız ve hızlı biçimde yürütülmesini sağlamıştır.

3.5. Sürüm Kontrolü

Proje boyunca Git kullanılmıştır. Her anlamlı işlev bir `commit` ile kaydedilmiş, commit mesajları Türkçe ve açıklayıcı biçimde yazılmıştır. `.gitignore` dosyası; derleme çıktıları (`build/`), Flutter'ın ürettiği ara dosyalar (`.dart_tool/`), Xcode'un kullanıcıya özel ayarları ve olası ortam değişkeni dosyaları depoya girmeyecek şekilde yapılandırılmıştır. Bu, hem depo boyutunu küçük tutmuş hem de kişisel yapılandırmaların paylaşılmasını önlemiştir.

4. SİSTEM MİMARİSİ

4.1. Katmanlı Mimari Yaklaşımı

Uygulama, sorumlulukların ayrılması (separation of concerns) ilkesine göre beş katmana bölünmüştür. Bu ayrımın amacı, bir katmandaki değişikliğin diğerlerini etkilememesidir.

Katman	Klasör	Sorumluluk
Model	lib/models/	Veri yapılarının tanımı ve JSON dönüşümleri. İş mantığı ve arayüz bilgisi içermez.
Servis	lib/services/	Dış dünyayla iletişim: HTTP istekleri, cihaz depolaması, GPS, bildirim, RSS ayrıştırma.
Durum	lib/state/	Uygulamanın ortak verisi ve filtreleri. Servisleri çağırır, sonucu tutar, dinleyicileri uyarır.
Ekran	lib/screens/	Tam sayfa arayüzler. Durum katmanını dinler ve kullanıcı etkileşimlerini iletir.
Bileşen	lib/widgets/	Birden çok ekranda kullanılan yeniden kullanılabilir arayüz parçaları.
Yardımcı	lib/utills/	Saf fonksiyonlar ve sabitler: tema, renk kodlama, coğrafi hesaplama, gün gruplama, şehir verisi.

Bu ayrımın pratik faydası şudur: AFAD API'si yanıt biçimini değiştirirse yalnızca `lib/services/deprem_servisi.dart` ve `lib/models/deprem.dart` düzenlenir; ekranların, bileşenlerin ve durum katmanının hiçbir satırına dokunulmaz. Aynı şekilde arayüzde bir tasarım değişikliği yapıldığında veri katmanı etkilenmez.

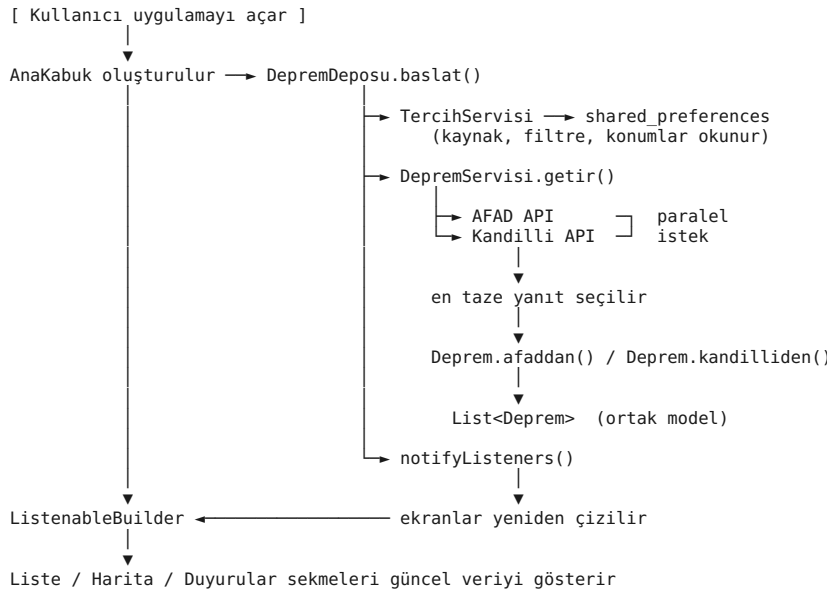
4.2. Proje Klasör Yapısı

lib/	
├── main.dart	Uygulama girişi, tema ve sistem çubuğu ayarı
├── models/	
│ ├── deprem.dart	Ortak deprem modeli + iki JSON çözümleyici
│ ├── kayitli_konum.dart	Kullanıcının takip ettiği yerler
│ ├── hazirlilik_maddesi.dart	Hazırlık listesi ve tamamlanma durumu
│ ├── acil_kisi.dart	Acil kişi modeli, telefon biçimleri
│ └── haber.dart	Haber modeli ve süzme kuralları
├── services/	
│ ├── deprem_servisi.dart	AFAD/Kandilli istekleri, kaynak seçimi, birleştirme
│ ├── tercih_servisi.dart	Cihaz depolaması sarmalayıcısı
│ ├── bildirim_servisi.dart	Yerel bildirim sarmalayıcısı
│ ├── konum_servisi.dart	GPS + basamaklı yedekleme
│ ├── acil_durum_servisi.dart	Mesaj üretimi, SMS/WhatsApp/arama kanalları
│ ├── haber_servisi.dart	RSS indirme ve ayrıştırma
│ ├── bulten_uretici.dart	Artçı sayımı ve bülten üretimi
│ └── hatirlatma_planlayici.dart	Sonraki hatırlatma tarihinin hesabı
├── state/	
│ └── deprem_deposu.dart	Ortak durum (ChangeNotifier) – 698 satır
├── screens/	14 tam sayfa ekran
├── widgets/	5 yeniden kullanılabilir bileşen
├── utills/	
│ ├── tema.dart	Renk paleti ve bileşen stilleri
│ ├── buyukluk_stili.dart	Büyükliğe göre renk / etiket / boyut
│ ├── siddet_hesabi.dart	Haversine mesafesi + MMI şiddet tahmini
│ ├── gun_gruplama.dart	Listeyi günlere ayırma
│ └── şehirler.dart	81 il koordinatı

Şekil 4.1 — Kaynak kodu dizin yapısı (ayrıntılı liste Ek A'dadır)

4.3. Veri Akışı

Uygulamanın açılışından ekranda liste görüntülenmesine kadar geçen akış aşağıdaki gibidir:



Şekil 4.2 — Açılıştan ekrana veri akışı

Uygulama ön plandayken `Timer.periodic` ile 60 saniyede bir yenileme tetiklenir. Bu aralık rastgele seçilmemiştir: Kandilli kaynağı yaklaşık 5 dakikalık gecikmeyle yayın yaptığından 60 saniye bu gecikmenin belirgin biçimde altında kalır ve yeni kayıt yayımlanır yayımlanmaz yakalanır. Daha sık yenileme, kaynağın sunucusuna gereksiz yük bindirecektir. Uygulama arka plana geçtiğinde zamanlayıcı durdurulur; arka planda istek atmak hem pil hem veri israfıdır ve kullanıcı sonucu görmemektedir.

4.4. Durum Yönetimi

Liste, harita ve duyurular sekmeleri aynı veriyi ve aynı filtreleri paylaşmaktadır. Her sekmenin kendi verisini çekmesi hâlinde iki sorun ortaya çıkardı: aynı veri için üç ayrı ağ isteği atılırdı ve listede uygulanan bir filtre haritaya yansımaya sekmeler arasında tutarsızlık oluştururdu.

Çözüm olarak `DepremDeposu` adında bir `ChangeNotifier` sınıfı yazılmıştır. Bu sınıfın tek örneği `AnaKabuk` içinde oluşturulur ve üç sekmeye de aynı örnek verilir. Durum değiştiğinde `notifyListeners()` çağrılır; ekranlar `ListenableBuilder` ile bu bildirimi dinleyip yalnızca ilgili bölümlerini yeniden çizer.

```

class DepremDeposu extends ChangeNotifier {
  DepremYaniti? _yaniti;
  bool yukleniyor = true;
  String? hata;

  List<Deprem> get _ham => _yaniti?.depremler ?? const [];

  Future<void> yenile() async {
    yukleniyor = true;
    hata = null;
    notifyListeners(); // ekranlar iskelet yüklemeye geçer
    try {
      _yaniti = await DepremServisi.getir(
        kaynak: kaynak, gunSayisi: gunSayisi, minBuyukluk: minBuyukluk,
      );
    } catch (e) {
      hata = e.toString();
    } finally {
      yukleniyor = false;
      notifyListeners(); // ekranlar sonucu gösterir
    }
  }
}

```

Provider, Riverpod veya BLoC gibi bir durum yönetimi paketi kullanılmamıştır. Bu tercih iki gerekçeye dayanır: bu ölçekteki bir uygulama için Flutter'ın kendi araçları yeterlidir ve dış paket kullanmamak, durum yönetiminin altında yatan mekanizmanın öğrenilmesini sağlamıştır.

4.5. Filtrelerin İkiye Ayrılması

Uygulamadaki filtreler, sunucuya gitme gereksinimlerine göre iki gruba ayrılmıştır. Bu ayrım, kullanıcı deneyimi ile ağ trafiği arasındaki dengeyi kurar.

Sunucuya Giden Filtreler	Cihazda Uygulanan Filtreler
Veri kaynağı (AFAD / Kandilli / Otomatik)	Hızlı filtreler (Yerlerim, Son 1 saat, Bugün...)
Zaman aralığı (gün sayısı)	Şehir/konum araması
Minimum büyüklük	Sıralama (en yeni / en büyük)
<i>Sol sütundaki bir değer değiştiğinde yeni bir HTTP isteği atılır. Sağ sütundakiler eldeki veri üzerinde anında uygulanır ve ağ trafiği oluşturmaz.</i>	

Bu ayrım sayesinde arama kutusuna yazılan her harf için API'ye istek gönderilmez. Benzer biçimde minimum büyüklük kaydırıcısında, `onChanged` olayı yalnızca ekrandaki etiketi güncellerken asıl istek parmak kaldırıldığında tetiklenen `onChangeEvent` olayında atılır. Bu iyileştirme yapılmadan önce kaydırıcı hareket ettirildiği sürece saniyede onlarca istek gönderiliyordu.

5. VERİ KAYNAKLARI VE API ENTEGRASYONU

5.1. AFAD Deprem Servisi

AFAD (Afet ve Acil Durum Yönetimi Başkanlığı), Türkiye'nin resmî afet kurumudur ve deprem kayıtlarını açık bir REST API üzerinden sunmaktadır.

```
GET https://deprem.afad.gov.tr/apiv2/event/filter
?start=2026-07-19T00:00:00
&end=2026-07-26T23:59:59
&orderby=timedesc
&limit=1000
&minmag=3.5
```

Servis, düz bir JSON dizisi döndürür. Entegrasyon sırasında dikkat edilmesi gereken en önemli nokta, **tüm sayısal alanların metin (string) olarak gönderilmesidir**. Örneğin büyüklük alanı `"magnitude": "3.7"` biçimindedir. Bu değerin doğrudan `double` tipine atanması çalışma zamanı hatası üretir; bu nedenle `double.TryParse` ile güvenli dönüşüm yapılır ve başarısızlık durumunda varsayılan değer kullanılır.

```
factory Deprem.afaddan(Map<String, dynamic> json) {
  return Deprem(
    id:      json['eventID']?.toString() ?? '',
    yer:     json['location']?.toString() ?? 'Bilinmeyen konum',
    buyukluk: double.TryParse(json['magnitude']?.toString() ?? '') ?? 0,
    derinlik: double.TryParse(json['depth']?.toString() ?? '') ?? 0,
    enlem:   double.TryParse(json['latitude']?.toString() ?? '') ?? 0,
    boylam:  double.TryParse(json['longitude']?.toString() ?? '') ?? 0,
    tarih:   DateTime.TryParse(json['date']?.toString() ?? '') ?? DateTime.now(),
    kaynak:  'AFAD',
  );
}
```

AFAD'ın önemli bir avantajı, tarih aralığı ve minimum büyüklük filtrelerini **sunucu tarafında** desteklemesidir. Bu sayede “son 30 günün 3.0 üzerindeki depremleri” gibi bir sorgu, geniş bir zaman aralığını kapsamasına rağmen yalnızca birkaç düzine kayıt döndürür. Bu özellik, duyurular sekmesindeki bülten üretimi için kritik önemdedir.

5.2. Kandilli Rasathanesi Verisi

Boğaziçi Üniversitesi Kandilli Rasathanesi ve Deprem Araştırma Enstitüsü, deprem verilerini yayımlamakla birlikte makine tarafından okunabilir resmî bir API sunmamaktadır. Bu nedenle veriye, bu kayıtları JSON biçiminde sunan üçüncü taraf bir açık API üzerinden erişilmektedir.

```
GET https://api.orhanaydogdu.com.tr/deprem/kandilli/live?limit=500
```

Yanıt `{"status": true, "result": [...]}` biçiminde sarmalanmış olarak gelir. Bu kaynakla çalışırken karşılaşılan en kritik ayrıntı, **koordinatların GeoJSON standardına göre sıralanmasıdır**. GeoJSON'da koordinat dizisi `[boylam, enlem]` sırasındadır; oysa harita kütüphanelerinin çoğu ve genel alışkanlık `[enlem, boylam]` sırasındır. Bu iki sıra karıştırıldığında işaretçiler harita üzerinde tamamen yanlış konumlara — çoğu zaman başka bir kıtaya — düşer.

```
factory Deprem.kandilliden(Map<String, dynamic> json) {
  final geo = json['geojson'] as Map<String, dynamic>;
  final koordinatlar = (geo?['coordinates'] as List?) ?? const <num>[0, 0];

  // DİKKAT: GeoJSON sırası [boylam, enlem] – alışılmışın TERSİ
  final double boylam = koordinatlar.isNotEmpty
    ? (koordinatlar[0] as num).toDouble() : 0;
  final double enlem = koordinatlar.length > 1
    ? (koordinatlar[1] as num).toDouble() : 0;

  // Kandilli tarihi "2026-07-25 23:49:05" biçiminde gönderiyor.
  // DateTime.tryParse boşluk yerine "T" beklediği için değiştiriliyor.
  final tarihMetni =
    (json['date_time']?.toString() ?? '').replaceFirst(' ', 'T');
  ...
}
```

Bu kaynağın önemli bir sınırı, **yalnızca son 24 saatlik veriyi sunmasıdır**. Yanıtın `metadata` alanındaki `date_starts` ve `date_ends` değerleri her zaman tam 24 saat arayla gelmekte ve talep edilen kayıt sayısı ne olursa olsun bu davranış değişmemektedir. Ayrıca servis tarih aralığı ve büyüklük parametrelerini kabul etmemekte, yalnızca “son N kayıt” döndürmektedir; filtreleme cihaz tarafında yapılmaktadır.

5.3. İki Kaynağın Karşılaştırılması

Kriter	AFAD	Kandilli (üçüncü taraf API)
Resmîlik	Doğrudan resmî kurum API'si	Kurum verisi, üçüncü taraf servis üzerinden
Tarih filtresi	Destekleniyor (sunucu tarafı)	Desteklenmiyor
Büyüklük filtresi	Destekleniyor (<code>minmag</code>)	Desteklenmiyor
Geçmiş kapsamı	Sınırsız (tarih aralığı verilebilir)	Yalnızca son 24 saat
Ölçülen tazelik	~11,5 saat gecikme (26 Tem. 2026 ölçümü)	~5 dakika gecikme
Sayısal alan tipi	Metin (dönüşüm gerekir)	Sayı
Koordinat biçimi	Ayrı <code>latitude</code> / <code>longitude</code> alanları	GeoJSON dizisi <code>[boylam, enlem]</code>
Tarih biçimi	2026-07-25T07:40:15	2026-07-25 23:49:05
Süreklilik riski	Düşük (resmî kurum)	Orta (üçüncü taraf servis kapanabilir)

5.4. Otomatik Kaynak Seçimi

Test aşamasında yapılan ölçümler, iki kaynağın güncelliğinin sabit olmadığını göstermiştir. 26 Temmuz 2026 tarihli bir ölçümde Kandilli'nin en yeni kaydı yaklaşık 5 dakikalık iken AFAD'ınki yaklaşık 11,5 saat geriden gelmekteydi. Ancak bu oran her zaman aynı yönde değildir; AFAD'ın daha hızlı olduğu durumlar da gözlemlenmiştir.

Kullanıcıyı “şu anda hangi kaynak daha hızlı?” sorusuyla baş başa bırakmak yerine, varsayılan olarak **Otomatik** modu geliştirilmiştir. Bu modda her iki kaynağa *paralel* istek gönderilir ve en yeni kaydı daha güncel olan yanıt kullanılır. İstekler eşzamanlı gittiği için ek gecikme oluşmaz. Kaynaklardan biri hata verirse diğeri kullanılır; ikisi birden erişilemezse kullanıcıya anlaşılır bir hata mesajı gösterilir.

```

final sonuclar = await Future.wait([
  dene(VeriKaynagi.kandilli),
  dene(VeriKaynagi.afad),
]);

final gecerli = sonuclar.whereType<DepremYaniti>().toList();
if (gecerli.isEmpty) {
  throw Exception('Hiçbir veri kaynağına ulaşılamadı. '
    'İnternet bağlantınızı kontrol edin.');
```

Neden ana listede birleştirme yapılmıyor?

Ana listede iki kaynaktan gelen kayıtları birleştirmek yerine tek bir kaynak seçilmektedir. Gerekçe: iki kurum aynı depremi biraz farklı büyüklük, derinlik ve koordinat değerleriyle raporlar. Binlerce kayıt üzerinde yapılacak eşleştirmede oluşacak bir hata, var olmayan bir depremi listeye ekleyebilir. Bir afet uygulamasında uydurma kayıt üretmek, eksik kayıt göstermekten daha zararlıdır.

5.5. Kayıt Tekilleştirme (Birleştirme)

Duyurular sekmesindeki bültenler farklı bir strateji izler. Burada eşik 3.0 büyüklüğünde olduğundan elde yalnızca birkaç düzine kayıt bulunur; hem hata riski düşüktür hem de sonuç gözle doğrulanabilir. Bu nedenle bültenlerde AFAD'ın 30 günlük derinliği ile Kandilli'nin 24 saatlik tazeliği birleştirilir: AFAD'ın gecikmesinden doğan boşluğu Kandilli tam olarak doldurur.

İki kaydın aynı depremi gösterip göstermediği iki koşulla belirlenir:

- Zaman farkı ≤ 120 saniye
- Merkez üssü mesafesi ≤ 50 km

Bu toleranslar, iki kurumun aynı depremi biraz farklı zaman ve koordinatla raporlamasını karşılayacak kadar geniş, ancak arka arkaya meydana gelen ayrı depremleri yanlışlıkla birleştirmeyecek kadar dardır. Çakışma hâlinde öncelikli listedeki (AFAD, resmî parametreler) kayıt korunur.

```

static const aynıDepremZamanToleransi = Duration(seconds: 120);
static const aynıDepremMesafeKm = 50.0;

static bool aynıDepremMi(Deprem a, Deprem b) {
  final zamanFarki = a.tarih.difference(b.tarih).abs();
  if (zamanFarki > aynıDepremZamanToleransi) return false;

  final mesafe = SiddetHesabi.mesafeKm(a.enlem, a.boylam, b.enlem, b.boylam);
  return mesafe <= aynıDepremMesafeKm;
}
```

Bu mantık `test/birlestirme_test.dart` dosyasında 17 ayrı testle doğrulanmıştır; testler yalnızca doğru birleştirmeyi değil, tolerans sabitlerinin makul aralıkta kaldığını da denetler.

5.6. Kapsama Şeffaflığı

Geliştirme sırasında fark edilen sessiz bir hata, ayrı bir sınıf yazılmasını gerektirmiştir. Kandilli kaynağı yalnızca “son N kayıt” döndürdüğü ve Türkiye’de günde ortalama 100–300 deprem kaydedildiği için, 1000 kayıtlık bir istek bile ancak 3–7 günü kapsayabilmektedir. Kullanıcı “son 30 gün” filtresini seçtiğinde liste dolu görüldüğü hâlde elde 30 günlük veri bulunmamaktaydı.

Bu, kullanıcının yanlış sonuç çıkarmasına yol açan sinsi bir eksikliktir: kişi “son 30 günde bu bölgede sadece 4 deprem olmuş” diye düşünebilir. Çözüm olarak servis, yalnızca liste döndürmek yerine kapsama bilgisini de içeren bir `DepremYanıt` nesnesi döndürecek biçimde yeniden yazılmıştır.

```
/// Veri istenen aralığı kapsamıyor mu?
bool get kapsamEksik {
  final enEski = enEskiKayıt;
  if (enEski == null) return false;
  if (depremler.length < kayıtSiniri) return false; // sınıra dayanmamış

  // 2 saatlik tolerans: küçük sapmalar doğal
  return enEski.isAfter(istenenBaslangic.add(const Duration(hours: 2)));
}

/// Kaynak belirgin biçimde gecikmeli mi?
bool get gecikmeli {
  final t = tazelik;
  return t != null && t > const Duration(hours: 3);
}
```

Arayüz bu bilgileri kullanarak kullanıcıya “Kandilli yalnızca son 24 saati sağlıyor, seçtiğiniz 7 günlük aralık karşılanamıyor” veya “Bu kaynak şu anda gecikmeli yayın yapıyor” gibi uyarılar gösterir. *Sessizce çalışmayan bir filtre, açıkça bozuk bir filtreden daha yanıltıcıdır.*

5.7. Haber Beslemesi ve Harita Servisi

TRT Haber RSS. Duyurular sekmesindeki haber katmanı, devlet yayıncısı TRT Haber’in RSS beslemelerinden beslenir. İki besleme birlikte kullanılır: son dakika beslemesi görsel adresini `<imageUrl>` etiketinde verirken manşet beslemesinde görsel yalnızca `<description>` içindeki HTML’e gömülü gelmektedir; bu nedenle iki farklı çıkarma yöntemi uygulanır. Anadolu Ajansı beslemesi de denenmiş ancak sıkıştırılmış olarak döndüğü ve güvenilir biçimde çözülmediği için kullanılmamıştır.

RSS tarihleri RFC 822 biçimindedir (Sun, 26 Jul 2026 12:43:00 +0300). Dart’ın `DateTime.tryParse` fonksiyonu bu biçimi tanımadığından ayrıştırma elle yazılmıştır.

OpenStreetMap. Harita döşemeleri `flutter_map` paketi üzerinden OpenStreetMap’ten alınmaktadır. Google Maps’ten farklı olarak API anahtarı ve faturalandırma hesabı gerektirmemesi, projeyi hem maliyetsiz hem de kurulum açısından basit kılmıştır.

Ortak HTTP Yardımcısı ve Türkçe Karakter Sorunu

Tüm ağ istekleri tek bir yardımcı fonksiyondan geçirilir. Bu fonksiyon zaman aşımını yönetir ve teknik hataları kullanıcının anlayabileceği Türkçe mesajlara çevirir. Ayrıca burada, geliştirme sürecinin en erken hatalarından biri çözülmüştür: yanıt gövdesi `response.body` ile okunduğunda Türkçe karakterler (ç, ğ, ı, ö, ş, ü) bozuk görünmekteydi. Doğru yöntem, ham baytların açıkça UTF-8 olarak çözülmesidir.

```
static Future<String> _istekAt(Uri adres, String kaynakAdi) async {
  try {
    final yanıt = await http.get(adres).timeout(_zamanAsimi);
    if (yanıt.statusCode != 200) {
      throw Exception('$kaynakAdi sunucusu ${yanıt.statusCode} hatası döndü.');
```

```
    }
    // ÖNEMLİ: yanıt.body yerine bodyBytes + utf8.decode
    return utf8.decode(yanıt.bodyBytes);
  } on TimeoutException {
    throw Exception('$kaynakAdi sunucusu zamanında yanıt vermedi.');
```

```
  } on FormatException {
    throw Exception('$kaynakAdi verisi okunamadı (bozuk yanıt).');
```

6. UYGULAMA MODÜLLERİ

6.1. Model Katmanı

Model katmanı, uygulamanın veri yapılarını tanımlar. Bu sınıflar iş mantığı içermez; yalnızca veriyi taşır ve JSON dönüşümlerini yapar.

Sınıf	Satır	İçerik
Deprem	109	Kimlik, yer, büyüklük, derinlik, enlem, boylam, tarih, kaynak alanları; <code>afaddan()</code> ve <code>kandilliden()</code> fabrika kurucuları; geçen süre, tarih metni, koordinat metni, harita bağlantısı ve paylaşım metni üreten yardımcı özellikler
KayıtlıKonum	82	Kullanıcının takip ettiği yer: ad, enlem, boylam, simge. JSON'a çevrilerek cihazda saklanır
HazırlıkMaddesi	228	Hazırlık maddesi tanımı (başlık, açıklama, periyot, bildirim metni, sabit bildirim kimliği) ve <code>HazırlıkDurumu</code> sınıfı (tamamlanma tarihleri, açık hatırlatmalar, hatırlatma saati)
AcilKişi	163	Acil kişi modeli; telefon numarası temizleme, doğrulama, WhatsApp biçimi ve görüntüleme biçimi dönüşümleri
Haber	139	Haber modeli; deprem ilgisi, Türkiye ilgisi ve tahmin iddiası tespiti için süzme kuralları

`Deprem` sınıfı mimarinin kilit noktasıdır: iki farklı API'den gelen iki farklı JSON şeması, tek bir ortak modele dönüştürülür. Bu sayede uygulamanın geri kalanı verinin hangi kaynaktan geldiğini bilmek zorunda kalmaz. Kaynak bilgisi yalnızca bir alan olarak taşınır ve arayüzde etiket göstermek için kullanılır.

6.2. Servis Katmanı

Servisler dış dünyayla olan tüm iletişimi kapsüller. Her servis tek bir sorumluluğa sahiptir.

Servis	Satır	Sorumluluk
DepremServisi	487	AFAD ve Kandilli istekleri, otomatik kaynak seçimi, bülten için hedefli sorgu, kayıt birleştirme, ortak HTTP yardımcısı ve hata çevirisi
TercihServisi	172	<code>shared_preferences</code> sarmalayıcısı: filtreler, kaynak, kayıtlı konumlar, acil kişiler ve hazırlık durumunun okunup yazılması
BildirimServisi	257	Yerel bildirim altyapısının başlatılması, izin isteme, zamanlanmış bildirim kurma ve iptal etme
KonumServisi	198	GPS okuma, izin durumu sorgulama, dört basamaklı yedekleme zinciri
AcilDurumServisi	175	Acil mesaj metninin üretilmesi; SMS, WhatsApp, telefon ve pano kanallarının açılması
HaberServisi	192	İki RSS beslemesinin indirilmesi, XML ayrıştırma, görsel adresi çıkarma, HTML temizleme, RFC 822 tarih çözümü
BültenUretici	269	Önemli depremlerin belirlenmesi, artçı sayımı ve eğilimi, kullanıcı konumlarındaki etki hesabı
HatırlatmaPlanlayıcı	138	Bir sonraki hatırlatma tarihinin saf fonksiyonla hesaplanması ve bildirimlerin toplu yeniden planlanması

6.3. Yardımcı Sınıflar

Sınıf	Satır	İçerik
SiddetHesabi	217	Haversine mesafesi, hiposantr mesafesi, Allen ve ark. (2012) MMI denklemi, kategori dönüşümü, geçerlilik denetimi
Sehirler	157	81 il merkezinin koordinatları ve konum türü simgeleri
Tema	159	Koyu tema renk paleti ve bileşen stilleri
GunGruplama	88	Kayıt listesinin günlere ayrılması ve “Bugün / Dün / 20 Temmuz” başlık metinlerinin üretilmesi
BuyuklukStili	39	Büyüklik değerine göre renk, etiket ve işaretçi boyutu eşlemesi

Bu sınıfların tamamı *saf fonksiyon* niteliğindedir: aynı girdi için her zaman aynı çıktıyı üretirler, dış duruma bağlı değildirler ve yan etkileri yoktur. Bu özellik, birim testi yazmayı son derece kolaylaştırmıştır — testler internet bağlantısı, cihaz veya kullanıcı etkileşimi gerektirmeden çalışır.

6.4. Ekranlar ve Bileşenler

Ekran	Satır	İşlev
ana_kabuk.dart	172	Alt sekme çubuğu, sekme yönetimi, DepremDeposu örneğinin oluşturulması, uygulama yaşam döngüsü izleme
liste_sekmesi.dart	651	Arama, hızlı filtreler, durum kartı, günlere ayrılmış liste, aşağı çekerek yenileme
harita_sekmesi.dart	327	OpenStreetMap haritası, işaretçiler, seçim kartı
detay_ekrani.dart	610	Deprem ayrıntıları, yerlerdeki tahmini etki, mini harita, kopyalama işlemleri
duyurular_sekmesi.dart	758	Haber kartları ve resmî veri bültenleri
acil_durum_ekrani.dart	775	Acil buton, kişi listesi, 112 kısayolu, “İyiyim” mesajı
hazirlik_ekrani.dart	551	Hazırlık kontrol listesi ve hatırlatma ayarları
ayarlar_sekmesi.dart	365	Kaynak seçimi, tercihler, hakkında
erken_uyari_ekrani.dart	359	Erken uyarı hakkında bilgilendirme ve resmî çözümlere yönlendirme
tanitim_ekrani.dart	354	İlk açılış tanıtımı (üç ekran)
konum_ekle_ekrani.dart	339	Şehir listesinden veya harita üzerinden yer seçme
yerlerim_ekrani.dart	269	Kayıtlı yerlerin listesi ve yönetimi
acilis_ekrani.dart	260	Animasyonlu giriş ekranı
acil_kisi_ekle_ekrani.dart	234	Acil kişi ekleme ve düzenleme

Bileşen katmanında beş yeniden kullanılabilir widget bulunur: `DepremKarti` (listedeki tek kayıt), `DurumKarti` (listenin üstündeki kişisel özet), `IskeletKart` (yükleme sırasındaki parıldayan taslak), `FiltrePaneli` (alttan açılan ayrıntılı filtreler) ve `DurumGorunumu` (boş ve hata ekranları).

7. HİSSEDİLİRLİK TAHMİNİ MODÜLÜ

Bu bölüm, projenin teknik açıdan en özgün ve en çok araştırma gerektiren kısmını anlatmaktadır.

7.1. Problemin Tanımı

Deprem büyüklüğü (magnitude), depremin kaynağında açığa çıkan enerjinin logaritmik bir ölçüsüdür. Bu değer *depremin kendisine* ait tek bir sayıdır ve gözlemcinin nerede bulunduğundan bağımsızdır. Buna karşılık *şiddet* (intensity), belirli bir noktada hissedilen sarsıntının ve oluşan etkilerin ölçüsüdür; her nokta için farklıdır.

Kullanıcıya yalnızca büyüklük gösterildiğinde, kendisini ilgilendiren sorunun cevabını hesaplama işi kullanıcıya bırakılmış olur — ki bu hesap uzmanlık gerektirir. Uygulamanın çözdüğü problem tam olarak budur: büyüklüğü, mesafeyi ve derinliği alıp kullanıcının bulunduğu noktadaki beklenen şiddete çevirmek.

7.2. Kullanılan Bilimsel Model

Kaynak

Allen, T. I., Wald, D. J. & Worden, C. B. (2012). *Intensity attenuation in active crustal regions*. Journal of Seismology, 16, 409–433.

Bu yayın, aktif kabuk bölgeleri (Türkiye de bu sınıfa girer) için geliştirilmiş bir **şiddet tahmin denklemi** (Intensity Prediction Equation) sunmaktadır. Yayında iki sürüm bulunur: biri fay yüzeyine olan en kısa mesafeyi (R_{rup}), diğeri odak noktasına olan mesafeyi (R_{hyp}) kullanır. Uygulamada **hiposantr mesafeli sürüm** tercih edilmiştir; çünkü açık veri API'leri fay geometrisi bilgisi sağlamamakta, yalnızca merkez üssü koordinatı ve derinlik vermektedir.

Denklem katsayıları, GEM Vakfı tarafından geliştirilen açık kaynaklı **OpenQuake Engine**'in AllenEtAl2012Rhypo uygulamasından alınmıştır. Bir bilimsel yayının denklemini kaynak koddan doğrulamak, katsayıların yanlış aktarılması riskini ortadan kaldırmıştır.

7.3. Mesafe Hesabı: Haversine Formülü

İlk adım, deprem merkez üssü ile kullanıcının kaydettiği nokta arasındaki yer yüzeyi mesafesini bulmaktır. Enlem ve boylam küresel koordinatlar olduğundan düzlemsel Öklid mesafesi kullanılamaz. Bunun yerine küre üzerindeki büyük daire mesafesini hesaplayan **haversine formülü** uygulanmıştır:

$$a = \sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2)$$

$$d = 2 \cdot R \cdot \arcsin(\sqrt{a})$$

ϕ : enlem (radyan) λ : boylam (radyan) $R = 6371,0088$ km

```
static double mesafeKm(double enlem1, double boylam1,
                      double enlem2, double boylam2) {
    double radyan(double derece) => derece * math.pi / 180.0;

    final f1 = radyan(enlem1);
    final f2 = radyan(enlem2);
    final dF = radyan(enlem2 - enlem1);
    final dL = radyan(boylam2 - boylam1);

    final a = math.sin(dF / 2) * math.sin(dF / 2) +
              math.cos(f1) * math.cos(f2) * math.sin(dL / 2) * math.sin(dL / 2);

    return 2 * _dunyaYaricapKm * math.asin(math.min(1.0, math.sqrt(a)));
}
```

Karekök sonucunun `math.min(1.0, ...)` ile sınırlandırılması bilinçlidir: kayan noktalı sayı hataları nedeniyle değer çok küçük bir farkla 1'i aşabilir ve `arcsin` fonksiyonu `NaN` döndürerek tüm hesabı bozar. Bu koruma `test/siddet_hesabi_test.dart` içinde ayrıca test edilmiştir.

7.4. Hiposantr Mesafesi

Yer yüzeyi mesafesi tek başına yeterli değildir. Depremin odak noktası (hiposantr) yer altında bulunduğundan, gerçek mesafe yüzey mesafesi ile derinliğin oluşturduğu dik üçgenin hipotenüsüdür:

$$R = \sqrt{d^2 + h^2}$$

d : yüzey mesafesi (km) h : odak derinliği (km)

Bu ayrım özellikle merkez üssüne yakın noktalarda belirleyicidir. 10 km derinlikte meydana gelen bir depremin tam üstünde bulunan bir nokta için yüzey mesafesi 0 km, hiposantr mesafesi ise 10 km'dir. Derinlik hesaba katılmasaydı sıfır mesafe, denklemden sonsuza giden bir değer üretecekti.

7.5. Şiddet Denklemi ve Katsayılar

Uygulanan denklem iki parçalıdır. 50 km'ye kadar temel denklem geçerlidir; bu mesafenin ötesinde ek bir sönümlenme terimi devreye girer.

$$R_m = m_1 + m_2 \cdot e^{(M - 5)}$$

$$MMI = c_0 + c_1 \cdot M + c_2 \cdot \ln(\sqrt{R^2 + R_m^2}) \quad R \leq 50 \text{ km}$$

$$MMI = c_0 + c_1 \cdot M + c_2 \cdot \ln(\sqrt{R^2 + R_m^2}) + c_4 \cdot \ln(R/50) \quad R > 50 \text{ km}$$

Katsayı	Değer	Açıklama
c_0	2.085	Sabit terim
c_1	1.428	Büyüklik katsayısı — büyüklik arttıkça şiddet artar
c_2	-1.402	Mesafe katsayısı — negatif olduğu için mesafe arttıkça şiddet azalır
c_4	0.078	50 km ötesi için ek sönümlenme düzeltmesi
m_1	-0.209	Doygunluk terimi sabiti
m_2	2.042	Doygunluk teriminin büyüklüğe bağlı bileşeni

R_m terimi *yakın alan doygunluğunu* temsil eder. Bu terim olmasaydı, mesafe sıfıra yaklaştıkça logaritma eksi sonsuza gider ve şiddet fiziksel olarak anlamsız biçimde sonsuza çıkardı. Gerçekte deprem enerjisi tek bir noktadan değil belirli bir fay yüzeyinden yayıldığı için şiddetin merkeze çok yakın bölgede bir üst sınıra doygunluğa ulaşması beklenir.

```
static double _hamMmi(double buyukluk, double hiposantrKm) {
    final rm = _m1 + _m2 * math.exp(buyukluk - 5.0);
    var deger = _c0 + _c1 * buyukluk +
        _c2 * math.log(math.sqrt(hiposantrKm * hiposantrKm + rm * rm));

    // 50 km'nin ötesinde ek bir mesafe terimi devreye giriyor
    if (hiposantrKm > 50.0) {
        deger += _c4 * math.log(hiposantrKm / 50.0);
    }
    return deger;
}
```

7.6. Sayıdan Kategoriye Dönüşüm

Denklem sonucu, Değiştirilmiş Mercalli Şiddet ölçeğinde (MMI) 1–12 aralığında bir ondalıklı sayıdır. Bu sayı kullanıcıya doğrudan gösterilmemektedir. Gerekçe: modelin belirsizliği, ondalık hassasiyeti anlamsız kılacak kadar büyüktür. “Şiddet: 4.37” gibi bir değer sahte bir kesinlik izlenimi yaratır.

Bunun yerine sonuç yedi kategoriye indirgenir ve her kategori günlük dille açıklanır:

MMI	Kategori	Kullanıcıya gösterilen açıklama
< 2.0	Hissedilmez	Bu konumda fark edilmesi beklenmiyor.
2.0 – 3.5	Zor hissedilir	Duran veya üst katlardaki bazı kişiler fark edebilir.
3.5 – 4.5	Hafif hissedilir	Bina içinde fark edilir; asılı eşyalar hafifçe sallanabilir.
4.5 – 5.5	Belirgin hissedilir	Çoğu kişi hisseder; eşyalar sallanır, bazıları uyanır.
5.5 – 6.5	Güçlü sarsıntı	Herkes hisseder; mobilyalar oynayabilir, sıva çatlayabilir.
6.5 – 7.5	Çok güçlü	Ayakta durmak zorlaşır; hasar görülebilir.
> 7.5	Şiddetli	Ciddi hasar beklenir.

7.7. Geçerlilik Sınırları ve Dürüstlük İlkesi

Kaynak yayında modelin geçerlilik aralığı M_W 5.0 – 7.9 ve 300 km olarak belirtilmiştir. Uygulamada gösterilen depremlerin büyük çoğunluğu 5.0'in altındadır; bu durumda model *ekstrapolasyon* yapmaktadır, yani eğitildiği veri aralığının dışında tahmin üretmektedir.

Bu durum gizlenmemiş, aksine üç farklı katmanda kullanıcıya iletilmiştir:

1. Sonuç ondalıklı sayı olarak değil, kaba kategori olarak gösterilir
2. Geçerlilik aralığı dışındaki tahminler “(tahmini)” etiketiyle işaretlenir
3. İlgili her ekranda, gerçek sarsıntının zemin yapısına, bina türüne ve kat yüksekliğine göre değişeceği belirtilir

```
final guvenilir = hiposantr <= gecerliMesafeKm      // 300 km
                && buyukluk >= gecerliMinBuyukluk; // 5.0

var mmi = _hamMmi(buyukluk, guvenliMesafe);
if (!mmi.isFinite) mmi = 1.0;      // NaN / sonsuz koruması
mmi = mmi.clamp(1.0, 12.0);      // ölçek dışına taşmayı engelle
```

Gizlilik notu

Kayıtlı konumlar yalnızca cihazda saklanır ve hiçbir sunucuya gönderilmez. Kullanıcı yerini 81 il listesinden veya harita üzerinden kendisi seçer; bu özellik için **konum izni dahi istenmez**. Tek istisna acil durum butonudur — o modül gerçek GPS gerektirir ve izin yalnızca butona basıldığı anda kullanılır. Arka planda takip veya sürekli dinleme yoktur.

7.8. Örnek Hesap

Aşağıdaki senaryoda, Malatya Akçadağ'da 11,5 km derinlikte meydana gelen 4.1 büyüklüğündeki bir depremin İstanbul'daki bir konutta hissedilirliği hesaplanmaktadır.

Adım	Sonuç
1. Yüzey mesafesi (haversine)	≈ 340 km
2. Hiposantr mesafesi $\sqrt{(340^2 + 11,5^2)}$	≈ 340,2 km
3. $R_m = -0,209 + 2,042 \cdot e^{(4,1-5)}$	≈ 0,621
4. Temel terim + 50 km ötesi düzeltmesi	MMI ≈ 1,1
5. Kategori dönüşümü (MMI < 2,0)	Hissedilmez
6. Geçerlilik denetimi (M 4,1 < 5,0)	Güvenilir = false → “(tahmini)” etiketi eklenir

Kullanıcıya gösterilen nihai metin: **“Evinizden 340 km — hissetmeyeceksiniz (tahmini)”**. Bu tek satır, uygulamanın varlık nedenini özetlemektedir: ham veriyi kullanıcının doğrudan kullanabileceği bir cevaba dönüştürmek.

8. BİLDİRİM VE HAZIRLIK MODÜLÜ

8.1. Üç Kavramın Ayrımı

Deprem uygulamaları söz konusu olduğunda birbirinden tamamen farklı üç kavram sıklıkla karıştırılmaktadır. Uygulama bu ayrımı kullanıcıya ayrı bir bilgilendirme ekranında açıkça anlatır.

Kavram	Bilimsel/Teknik Durum	Bu Uygulama
Deprem tahmini	Bilimsel olarak mümkün değil	Yapmıyor, iddia etmiyor
Erken uyarı	Gerçek, ancak sismik istasyon ağı gerektirir	Yapamıyor — kullanıcıyı gerçek çözümlere yönlendiriyor
Hazırlık hatırlatması	Tamamen uygulanabilir	Yapıyor

Tahmin neden mümkün değildir? USGS (Amerika Birleşik Devletleri Jeoloji Araştırmaları Kurumu), bugüne kadar hiçbir bilim insanının büyük bir depremi önceden bildiremediğini ve öngörülebilir gelecekte de bunun beklenmediğini açıkça belirtmektedir. Deprem tahmini vaat eden bir uygulama, kullanıcıyı yanıltmış olur.

8.2. Erken Uyarı Neden Yapılamıyor

Erken uyarı sistemleri, hızlı ilerleyen P dalgası (5–7 km/sn) ile yıkıcı S dalgası (3–4 km/sn) arasındaki hız farkından yararlanır. Sismik istasyonlar P dalgasını algıladığı anda, veri ışık hızında iletilerek S dalgası ulaşmadan önce uyarı gönderilir. Bu, saniyelerle ölçülen bir zaman kazancıdır.

Böyle bir sistem için sismik istasyon ağına ya da işletim sistemi düzeyinde erişime ihtiyaç vardır. Bu proje AFAD ve Kandilli'nin açık verilerini kullanmaktadır; bu veriler deprem gerçekleşip analiz edildikten *sonra* yayımlanır. Test sırasında AFAD'ın verisini saatler sonra yayımladığı ölçülmüştür — erken uyarı için dakikalar bile çok geçtir.

Sahte bir erken uyarı vaadi vermek yerine uygulama, kullanıcıya Android'in yerleşik deprem uyarılarını, iOS'un resmî uyarı ayarlarını ve AFAD Acil uygulamasını nasıl etkinleştireceğini adım adım anlatan bir ekran sunmaktadır.

8.3. Hatırlatma Planlama Algoritması

Uygulamada beş hazırlık maddesi tanımlıdır (deprem çantası kontrolü, ev içi güvenlik gözden geçirmesi, aile tatbikatı vb.). İçerik, AFAD ve Kızılay'ın yaygın olarak yayımladığı hazırlık önerilerine dayanmaktadır.

Teknik zorluk şudur: mobil işletim sistemlerinin yerel bildirim altyapısı “her 3 ayda bir” gibi uzun periyotları kendi başına tekrarlayamaz. Desteklenen tekrar aralıkları günlük, haftalık gibi kısa periyotlarla sınırlıdır.

Çözüm olarak *yeniden planlama* yaklaşımı benimsenmiştir:

- Her madde için yalnızca **bir sonraki** bildirim tarihi hesaplanır ve kurulur
- Uygulama her açıldığında tüm planlama iptal edilip yeniden kurulur

Bu yaklaşımın avantajı, kullanıcı uygulamayı hiç açmasa bile planlanmış bildirimin gitmesidir; kullanıcı uygulamayı açtığında ise bir sonraki bildirim kurulur. Ek altyapı, sunucu veya arka plan servisi gerektirmez.

Kontrol listesi hatırlatmaları besler: bir madde işaretlendiğinde tamamlanma tarihi kaydedilir ve periyot dolduğunda hatırlatma gönderilir. Hiç işaretlenmemiş maddeler için ise 3 gün sonra ilk dürtme gönderilir — hemen bildirim göndermek rahatsız edici, 90 gün beklemek ise anlamsız olurdu.

```
/// Bir maddenin bir sonraki hatırlatma zamanı.
/// Saf fonksiyon: test edilebilmesi için [simdi] dışarıdan verilebilir.
static DateTime? sonrakiZaman(
    HazirlikMaddesi madde, DateTime? sonTamamlanma, int saat,
    {required DateTime simdi}) { ... }
```

Fonksiyonun saf tutulması ve “simdi” değerinin parametre olarak alınması bilinçli bir karardır: bu sayede zaman bağımlı davranış, sistem saatini değiştirmeye gerek kalmadan test edilebilmektedir.

8.4. Bildirim Kimliği Çakışması

Yerel bildirimler tam sayı bir kimlikle tanımlanır. İlk uygulamada bu kimlik madde kimliğinin `hashCode` değerinden türetilmişti. Bu yaklaşımın iki ciddi sorunu tespit edilmiştir:

- Dart'ın `String.hashCode` değeri platformlar ve sürümler arasında aynı olmak zorunda değildir. Değer değişirse, önceden planlanmış bildirimler **iptal edilemez** hâle gelir ve kullanıcı kapattığı bir hatırlatmayı almaya devam eder.
- Hash değeri modulo ile daraltıldığında çakışma ihtimali doğar (5 madde için yaklaşık %0,1) ve çakışan iki hatırlatmadan biri diğerini ezer.

Çözüm olarak her maddeye **elle atanmış sabit tam sayı kimlikler** verilmiştir. Ayrıca `test/hatırlatma_test.dart` içine, kimliklerin birbirinden farklı olduğunu doğrulayan bir test eklenmiştir; ileride yeni bir madde eklendiğinde çakışma olursa test bunu derhal yakalayacaktır.

8.5. Paket Sürüm Seçimi

`flutter_local_notifications` paketi, ana sürümler arasında API'sini sık değiştirmektedir. Proje geliştirilirken karşılaşılan durum aşağıdaki tabloda özetlenmiştir.

Sürüm	Gereksinim	Fark
21.x	Flutter 3.38+	Projenin hedef sürümünden çok yeni
20.x	Flutter 3.32+	Tüm parametreler adlandırılmış parametreye çevrildi
19.x	Flutter 3.22+	Seçilen — API 19.x boyunca kararlı
18.x	Flutter 3.13+	<code>uiLocalNotificationDateInterpretation</code> parametresi hâlâ zorunlu

Bu nedenle bağımlılık `^19.0.0` olarak sabitlenmiştir. Caret (^) işareti 20.0.0'a geçişe izin vermediği için API şeklinin kararlılığı garanti altına alınmıştır.

Zamanlama `AndroidScheduleMode.inexactAllowWhileIdle` kipiyle yapılmaktadır. Bu kip `SCHEDULE_EXACT_ALARM` iznini gerektirmez; sistem bildirimi pil dostu bir zamanda gönderir. Hazırlık hatırlatmalarında dakika hassasiyeti gerekmediğinden bu doğru seçimdir — “deprem çantanı kontrol et” bildiriminin 14:00 yerine 14:20'de gelmesinin hiçbir sakıncası yoktur, buna karşılık kesin zamanlama izni istemek kullanıcı için gereksiz bir izin talebi olurdu.

9. ACİL DURUM MODÜLÜ

Acil durum modülü, kullanıcının deprem sonrasında yakınlarına hızla konum bilgisi içeren bir mesaj gönderebilmesini sağlar. Butona 1,5 saniye basılı tutulduğunda, konum bilgisi eklenmiş hazır bir mesaj SMS uygulamasında alıcılarla birlikte açılır.

9.1. Basamaklı Konum Yedeklemesi

Acil durumda konumun alınamaması, mesajın hiç gönderilememesine yol açmamalıdır. Bu ilkeye dayanarak dört basamaklı bir yedekleme zinciri tasarlanmıştır:

Sıra	Kaynak	Devreye girme koşulu
1	GPS (en yüksek doğruluk)	İzin verilmiş ve konum servisi açık — 12 saniye zaman aşımı
2	Cihazın son bilinen konumu	GPS zamanında yanıt vermezse — anında döner
3	Kayıtlı “Yerlerim” konumu	Hiç konum izni verilmemişse
4	Konumsuz mesaj	Hiçbiri kullanılamıyorsa — mesaj yine de gönderilir

GPS için belirlenen 12 saniyelik zaman aşımı bilinçli bir dengedir: acil durumda uzun süre beklemek anlamsızdır; 12 saniyede sonuç gelmezse yedeğe geçip mesajı göndermek daha değerlidir.

Konumun hangi kaynaktan geldiği hem arayüzde hem de **mesajın içinde** belirtilir. Alıcının, gelen koordinatın anlık GPS konumu mu yoksa daha önce kaydedilmiş bir adres mi olduğunu bilmesi kritik önemdedir; bu bilgi arama kurtarma açısından tamamen farklı anlamlar taşır.

9.2. İletişim Kanalları

Kanal	Davranış ve gerekçe
SMS	Önerilen birincil kanal. Deprem anında mobil veri ve Wi-Fi altyapısı çökerken SMS altyapısı genellikle daha dayanıklı kalır; internet gerektirmeyen tek kanaldır. Alıcılar ve mesaj dolu olarak SMS uygulaması açılır.
WhatsApp	WhatsApp çoklu alıcıyı desteklemediği için her kişi ayrı ayrı açılır; arayüzde bu nedenle kişi başına buton gösterilir.
112 araması	Arama ekranı numara yazılmış olarak açılır, otomatik arama yapılmaz.
Pano	Tüm kanallar başarısız olursa mesaj panoya kopyalanır ve kullanıcı bilgilendirilir. Her koşulda çalışan son yedek.

Mesaj metni bilinçli olarak kısa tutulmuştur: SMS 160 karakterde bölünür ve uzun mesajlar şebeke yoğunluğunda gecikebilir. Ayrıca sarsıntı sonrası yakınları rahatlatmak için ayrı bir “İyiymi” mesajı seçeneği bulunur.

9.3. Platform Kısıtları ve Şeffaflık

Uygulama mesajı kendiliğinden gönderemez

Ne iOS ne de Android, bir uygulamanın kullanıcı onayı olmadan SMS göndermesine izin vermektedir. Android’de `SEND_SMS` izni teknik olarak mevcuttur; ancak Google Play Store, varsayılan SMS uygulaması olmayan uygulamaları bu izin nedeniyle reddetmektedir. Son “gönder” dokunuşu her koşulda kullanıcıdadır.

Bu kısıt arayüzde açıkça yazılmıştır. Gerekçesi doğrudan güvenlikle ilgilidir: acil durumda kullanıcının “mesaj gitti” sanarak yardım beklemesi gerçek zarar doğurur. Bir uygulamanın yapamadığı şeyi gizlemesi, bu bağlamda kabul edilebilir bir tasarım tercihi değildir.

9.4. Yanlışlıkla Tetiklenmeye Karşı Koruma

Acil durum butonu tek dokunuşla değil, **1,5 saniye basılı tutmayla** tetiklenir ve bu süre boyunca dairesel bir ilerleme göstergesi görüntülenir. Kullanıcı isterse parmağını kaldırarak işlemi iptal edebilir.

Bu kararın gerekçesi yalnızca kullanıcı rahatsızlığı değildir. Aileye kazara acil durum mesajı gitmesi “kurt masalı” etkisi yaratır: bir sonraki gerçek uyarı ciddiye alınmayabilir. Yanlış alarmların maliyeti, bir buçuk saniyelik ek etkileşim maliyetinden çok daha yüksektir.

9.5. Android Paket Görünürlüğü Sorunu

Geliştirme sırasında karşılaşılan en sinsi hatalardan biri burada ortaya çıkmıştır. Android 11 ile birlikte gelen *paket görünürlüğü* (package visibility) kuralları gereği, bir uygulamanın başka uygulamaları sorgulayabilmesi için kullanacağı şemaları `AndroidManifest.xml` içindeki `<queries>` bloğunda önceden bildirmesi zorunludur.

Bu bildirim yapılmadığında `canLaunchUrl()` fonksiyonu **her zaman false döndürür** ve buton hiçbir hata mesajı üretmeden çalışmaz. Bir acil durum özelliğinde bu, olabilecek en kötü hata türüdür: sistem sessizce başarısız olur.

```
<queries>
  <!-- Flutter şablonunda zaten bulunan PROCESS_TEXT bildirimi -->
  <intent>
    <action android:name="android.intent.action.PROCESS_TEXT"/>
    <data android:mimeType="text/plain"/>
  </intent>

  <!-- Acil durum modülü için eklenenler -->
  <intent><action android:name="android.intent.action.VIEW"/>
    <data android:scheme="sms"/></intent>
  <intent><action android:name="android.intent.action.VIEW"/>
    <data android:scheme="tel"/></intent>
  <intent><action android:name="android.intent.action.VIEW"/>
    <data android:scheme="https"/></intent>
</queries>
```

Ayrıca dikkat edilmesi gereken bir ayrıntı daha vardır: Flutter'ın proje şablonunda `PROCESS_TEXT` için zaten bir `<queries>` bloğu bulunmaktadır. İkinci bir blok eklemek geçersizdir; yeni şemalar mevcut bloğun içine yazılmalıdır.

10. DUYURULAR MODÜLÜ

10.1. Neden Bülten Üretimi

Duyurular sekmesinin ilk tasarımı, deprem etiketli haberleri toplamak üzerine kuruluydu. İki sorun bu yaklaşımı değiştirmeyi gerektirdi.

Birinci sorun — makine okunabilir resmî duyuru kaynağı yok. AFAD'ın /rss adresi boş yanıt döndürmektedir; Kandilli'nin de yapılandırılmış bir duyuru beslemesi bulunmamaktadır. HTML sayfalarını kazımak (scraping) hem kırılgan bir çözümdür hem de kullanım koşulları açısından tartışmalıdır.

İkinci ve daha önemli sorun — tahmin iddiaları. Türkiye'de “deprem” etiketli haberlerin kayda değer bir bölümü “Uzman uyardı: X ilinde 7'lik deprem” türündedir. Uygulamanın başka bir ekranında deprem tahmininin bilimsel olarak mümkün olmadığı anlatılırken, filtresiz bir haber akışı sunmak bu mesajla doğrudan çelişir ve bir güvenlik uygulamasında panik yaratabilir.

Bu nedenle duyurular, **zaten kullanılmakta olan resmî veriden** üretilmektedir. Böylece dış bir kaynağa bağımlılık ve kırılma noktası oluşmaz.

10.2. Artçı Sayımı ve Eğilim

Her bülten için hesaplanan bilgiler:

- **Deprem parametreleri** — büyüklük, derinlik, koordinat; doğrudan AFAD/Kandilli verisinden
- **Artçı sayısı** — ana depremden sonra meydana gelen, daha küçük büyüklükte ve yaklaşık 100 km yarıçap içindeki kayıtlar
- **Artçı eğilimi** — son 6 saatteki artçı sayısı, önceki 6 saattekinden az mı?
- **Kullanıcının yerlerindeki tahmini etki** — 7. bölümdeki şiddet modeliyle hesaplanır

Etiketlemede önemli bir ayrıntı

Bülten verisi 3.0 büyüklük eşiğiyle çekildiği için, eşiğin altındaki artçılar veri setinde hiç bulunmaz; gerçek artçı sayısı hesaplanandan çok daha fazladır. Bu nedenle arayüzde “yaklaşık N artçı” **denmemekte**, bunun yerine “**3.0+ artçı: N**” yazılmaktadır. Aksi hâlde kullanıcı gerçek artçı sayısını olduğundan çok düşük sanacaktı.

Ayrıca eşiği geçen hiç deprem bulunmadığı durumda, listedeki en büyük kayıtlar yedek olarak gösterilir ancak farklı bir etiketle sunulur; kullanıcının sıradan bir sarsıntıyı “önemli duyuru” sanmaması sağlanır.

10.3. Haber Süzme Katmanları

Duyurular sekmesi iki katmanlıdır: resmî veriden üretilen bültenler kalıcı omurgayı oluştururken, TRT Haber beslemesinden gelen haberler olay yaşandığında devreye giren zenginleştirme katmanıdır.

Bu iki katmanlı yapının gerekçesi test sırasında ortaya çıkmıştır: TRT'nin manşet beslemesindeki 50 haberin **hiçbiri** deprem haberi değildi — çünkü o hafta meydana gelen en büyük deprem 4.1 büyüklüğündeydi. Haber katmanı tek başına kullanılsaydı sekme çoğu zaman boş kalacaktı.

Haberler üç aşamalı bir süzgeçten geçirilir:

Süzgeç	Çalışma biçimi
Deprem ilgisi	Başlık ve özetle deprem ile ilgili anahtar kelimeler aranır. Türkçe karakter farkı sorun oluşturmaz: “artçı” ve “artıcı” yazımlarının ikisi de yakalanır.
Türkiye ilgisi	Önce TRT kategorisine bakılır (“Türkiye” veya “Gündem” ise doğrudan kabul edilir). Ardından yabancı ülke adı aranır; varsa ve Türkiye işareti yoksa haber elenir. Böylece “Japonya’da deprem” elenirken “Yunanistan’da deprem, Türkiye’de de hissedildi” korunur.
Tahmin iddiası	“kâhin tarih verdi”, “ne zaman olacak” gibi kalıplar tespit edilir. Bu haberler gizlenmez, işaretlenir : kartın üstünde “Tahmin iddiası içerebilir — deprem önceden bilinemez” uyarısı görüntülenir.

Son süzgecin tasarımı bilinçli bir etik tercihtir. Haberi tamamen gizlemek sansür anlamına gelirdi; bağlam eklemek ise kullanıcıyı bilgilendirir ve karar yetkisini kendisine bırakır.

10.4. Telif ve Etik

RSS beslemeleri sendikasyon (yeniden yayım) amacıyla yayımlanır. Buna rağmen kullanım sınırlı tutulmuştur: yalnızca başlık, kısa özet, kaynak adı ve özgün bağlantı gösterilir; haberin tam metni kopyalanmaz. Görseller beslemenin kendi sağladığı adreslerden yüklenir. Karta dokunulduğunda haber, kaynağın kendi sitesinde açılır — böylece trafik ve reklam geliri kaynağa kalır.

11. ARAYÜZ VE KULLANICI DENEYİMİ TASARIMI

11.1. Görsel Dil ve Tema

Uygulama yalnızca koyu tema ile tasarlanmıştır ve açık tema seçeneği sunmaz. Bu kararın üç gerekçesi vardır: deprem ve afet uygulamalarının yaygın görsel dili koyu zemindir; büyüklüğü kodlayan renkler koyu zemin üzerinde belirgin biçimde daha okunaklı çıkar; ve deprem uygulamalarına sıklıkla gece bakıldığı için koyu zemin göz yormaz.

Renk paleti ve bileşen stilleri `lib/utls/tema.dart` içinde merkezî olarak tanımlanmıştır. Bu sayede bir renk değişikliği tek dosyadan yapılabilir ve uygulama genelinde tutarlılık korunur.

11.2. Bilgi Hiyerarşisi

Liste kartlarında göz önce sol taraftaki renkli büyüklük rozetine, ardından konuma, en son derinlik ve zaman gibi ayrıntılara yönelecek biçimde bir hiyerarşi kurulmuştur. Kartın sol kenarındaki renk şeridi, uzun listelerin hızlı taranmasını sağlar.

Filtrelerde de benzer bir ilke uygulanmıştır: sık kullanılan hızlı filtreler ana ekranda görünür durumda tutulurken, kaynak seçimi ve sıralama gibi daha seyrek değiştirilen ayarlar alttan açılan bir panele taşınmıştır. Ana ekran böylece sade kalır. Ayrıca kaç filtrenin etkin olduğu, filtre butonunun üzerinde bir sayı rozetiyle gösterilir; kullanıcı “neden az sonuç var?” sorusunun cevabını arayüzde doğrudan görebilir.

Cevabın en üste taşınması

Bir kullanıcının bu uygulamayı açmasının temel nedeni tek bir sorudur: “*Son 24 saatte benim yerlerimde bir şey oldu mu?*” Bu nedenle listenin en üstüne kişisel bir durum kartı yerleştirilmiştir. Kartın üç hâli vardır: kayıtlı yer yoksa ekleme çağrısı, hareket yoksa yeşil onay (“Yerleriniz sakın — son 24 saatte hissedilmesi beklenen deprem yok”), sarsıntı varsa en güçlüsünün özeti. Kullanıcı 200 kartlık bir listeyi taramak zorunda kalmaz.

Gün başlıkları

Düz bir liste, kullanıcının zaman algısını kaybetmesine yol açıyordu. Kaydırırken üste yapışan “Bugün / Dün / 20 Temmuz” başlıkları eklenerek liste taranabilir hâle getirilmiştir. Sıralama büyüklüğe göre değiştirildiğinde günlere göre gruplama anlamını yitirdiği için otomatik olarak düz listeye dönlür.

11.3. Algılanan Performans

Gerçek performans kadar *algılanan* performans da kullanıcı deneyimini belirler. Bu doğrultuda alınan kararlar:

Teknik	Etkisi
İskelet yükleme	Dönen çark yerine parıldayan kart taslakları gösterilir. Kullanıcı ne geleceğini gördüğü için bekleme daha kısa hissedilir; ekran dolduğunda düzen sıçraması olmaz.
Açılış animasyonunun işlevselleştirilmesi	Yaklaşık 2,2 saniyelik açılış animasyonu boyunca arka planda tercihler okunur ve ilk veri çekilir. Kullanıcı animasyonu izlerken liste hazırlanır; bekleme süresi boşa gitmez.
IndexedStack	Sekmeler arası geçişte widget'ların yeniden oluşturulmasını engeller. Haritanın konumu ve listenin kaydırma pozisyonu korunur.
Dokunsal geri bildirim	Filtre seçimi, sekme değişimi, yenileme ve konum kaydetme anlarında farklı şiddetlerde titreşim verilir. Bilinçli olarak fark edilmez ancak uygulamanın “bitmiş ürün” hissini güçlendirir.
Yönlendirici boş ekranlar	Her boş ve hata ekranı üç soruya cevap verir: ne oldu, neden oldu, şimdi ne yapılabilir. Bu nedenle hepsinde bir eylem butonu bulunur (örneğin hata durumunda “diğer kaynağı dene”).

İlk açılış tanıtımı

Geliştirmenin ortalarında fark edilen önemli bir sorun, uygulamanın en değerli özelliğinin (hissedilirlik tahmini) ayarların içinde gizli kalmasıydı. *Görünmeyen bir özellik, olmayan bir özellikle aynı şeydir.* Bu nedenle ilk açılışta üç ekranlık bir tanıtım eklenmiş ve kullanıcı ilk yerini eklemeye yönlendirilmiştir. Tanıtım atlanabilir tutulmuştur; zorunlu onboarding kullanıcı kaybettirir.

11.4. Erişilebilirlik

Erişilebilirlik, geliştirme sonuna eklenen bir katman olarak değil, tasarımın başından itibaren bir gereksinim olarak ele alınmıştır.

Uygulanan önlem	Kimin için
Büyüklik yalnızca renkle değil, sayıyla da kodlanır (harita işaretçileri dahil)	Renk körlüğü olan kullanıcılar bilgi kaybetmez
Kartlar tek bir <code>Semantics</code> düğümü olarak anlamlı bir cümle hâlinde okunur	Ekran okuyucu kullanıcıları parça parça değil, bütün cümle duyar
Rozetlerdeki sayılar <code>FittedBox</code> içine alınmıştır	Büyük yazı tipi ayarında metin taşmaz
Tüm dokunma hedefleri en az 48×48 piksel	Motor becerisi kısıtlı kullanıcılar ve küçük ekranlar
Renk kontrast oranları koyu zemine göre seçilmiştir	Düşük görme keskinliği ve güçlü ışık altında kullanım

11.5. Uygulama Kimliği: İkon ve Açılış

Uygulama ikonu setleri, 512×512 boyutundaki kaynak görselden (`assets/logo.png`) programatik olarak üretilmiştir.

Platform	Üretilen dosyalar
iOS	15 farklı boyut ve <code>Contents.json</code> (20pt–1024pt). Şeffaflık ve köşe yuvarlatması uygulanmaz; sistem kendi maskesini kullanır.
Android (klasik)	5 yoğunlukta <code>ic_launcher.png</code> ve yuvarlak maskeli <code>ic_launcher_round.png</code> (48–192 piksel)
Android (uyarlanabilir)	5 yoğunlukta <code>ic_launcher_foreground.png</code> ve arka plan renk kaynağı. İçerik %64 oranında ölçeklenip ortalanmıştır; sistem daire, kare veya damla maskesi uyguladığında kırılma olmaz.

Kaynak görselde tespit edilen sorun: ikon yuvarlatılmış kare olarak tasarlanmış ve köşelerin dışı beyaz bırakılmıştı. iOS kendi maskesini uyguladığı için bu beyaz kenar görünür kalacaktı. Çözüm olarak köşelerden başlayan bir *flood fill* işlemiyle beyaz alan ikonun kendi zemin rengiyle doldurulmuştur. İkon içindeki beyaz ev şekli, dış beyaz bölgeye bağlı olmadığı için bu işleminden etkilenmemiştir.

Beyaz parlama sorunu: İşletim sisteminin gösterdiği yerel açılış ekranı (Android'de `launch_background.xml`, iOS'ta `LaunchScreen.storyboard`) varsayılan olarak beyazdır. Koyu temalı bir uygulamada bu, her açılışta göz alan bir parlama anlamına gelir. Her iki platformun açılış ekranı zemini de logonun kendi zemin rengiyle (`#0B131B`) eşitlenmiş, böylece yerel açılış ekranından Flutter arayüzüne geçiş dikişsiz hâle getirilmiştir.

Açılış animasyonu: Logodaki sismik halkalar sabit bir görüntüdür; animasyonda bu halkalar logonun dışına doğru yayılan canlı halkalarla sürdürülmüştür. Görsel dil böylece statik ikondan hareketli açılışa kesintisiz aktarılmıştır.

12. TEST VE KALİTE GÜVENCESİ

12.1. Test Stratejisi

Projede **143 birim testi**, 33 test grubu içinde 7 dosyaya dağıtılmış durumdadır. Testler `flutter test` komutuyla çalıştırılır.

Test kapsamı bilinçli olarak model ve mantık katmanlarına odaklanmıştır; arayüz testleri (widget testleri) yazılmamıştır. Bu tercihin gerekçeleri:

- **Hata riskinin yoğunlaştığı yer burasıdır.** İki farklı API'nin JSON çözümlemesi, koordinat sırası, tarih biçimi, matematiksel hesaplar ve süzme kuralları — hataların sessizce oluşup yanlış bilgi üretebileceği alanlar bunlardır.
- **Arayüz hataları görünürdür.** Bir düğmenin yanlış yerde durması uygulamayı çalıştırınca hemen fark edilir; oysa koordinat sırası hatası, işaretçi yanlış yere düşene kadar sessizce ilerler.
- **Testler ağ bağımsızdır.** Test edilen katmanların tamamı saf fonksiyon niteliğinde olduğundan testler internet bağlantısı, cihaz veya emülatör gerektirmeden saniyeler içinde çalışır.
- **Arayüz sık değişir.** Geliştirme boyunca tasarım defalarca güncellenmiştir; arayüz testleri her değişiklikte bakım maliyeti yaratacaktı.

12.2. Test Kapsamı

Dosya	Test	Kapsanan konular
widget_test.dart	15	AFAD'ın metin biçimindeki sayıları, Kandilli'nin GeoJSON koordinat sırası, eksik ve bozuk veriye dayanıklılık, görüntüleme yardımcıları
siddet_hesabi_test.dart	20	Haversine mesafesi, bilinen deprem senaryoları, monotonluk (mesafe artınca şiddet azalmalı), sınır durumları, geçerlilik bayrağı, seviye eşikleri
acil_durum_test.dart	31	Telefon numarası temizleme ve doğrulama, WhatsApp biçimi, görüntüleme biçimi, mesaj üretimi, konumsuz durum, kişi saklama, konum sonucu
hatirlatma_test.dart	26	Hazırlık listesi tutarlılığı, sonraki hatırlatma tarihi, geçmişe planlama koruması, gecikmiş kontrolü, tamamlanma metni, durum saklama
haber_test.dart	20	Deprem konusu tespiti, Türkiye ilgisi, tahmin iddiası tespiti, süzme ve sıralama, haber modeli
birlestirme_test.dart	17	Aynı deprem tespiti, birleştirme mantığı, tolerans sabitlerinin makullüğü
gun_gruplama_test.dart	14	Temel gruplama, gece yarısı sınırı, yıl geçişi, boş liste, başlık metinleri, kayıt kaybolmaması
TOPLAM	143	33 test grubu

12.3. Örnek Test Senaryoları

Testlerin bir bölümü yalnızca “doğru sonucu veriyor mu?” sorusunu değil, **matematiksel ve mantıksal tutarlılığı** sorgular. Bu tür testler, formül yanlış yazıldığında sonucun tesadüfen doğru çıkmasını da engeller.

Test	Doğrulan özellik
Monotonluk testi	Aynı deprem için mesafe arttıkça tahmini şiddet <i>her zaman</i> azalmalıdır. Bu, denklemdaki mesafe katsayısının işaretini doğrular.
Sıfır mesafe testi	Kullanıcının tam merkez üssünde bulunması hâlinde hesap NaN veya sonsuz üretmemelidir.
Negatif derinlik testi	Bozuk veride negatif derinlik gelirse hesap çökmemeli, derinlik sıfır kabul edilmelidir.
Yıl geçişi testi	31 Aralık – 1 Ocak sınırında gün gruplaması doğru çalışmalı, kayıtlar kaybolmamalıdır.
Kayıt kaybolmaması testi	Gruplama işlemi sonrası toplam kayıt sayısı, giriş listesinin uzunluğuna eşit olmalıdır.
Bildirim kimliği çakışma testi	Tanımlı tüm hazırlık maddelerinin bildirim kimlikleri birbirinden farklı olmalıdır.
Geçmişe planlama testi	Hesaplanan hatırlatma tarihi geçmişte kalıyorsa bildirim ileri bir tarihe ertelenmelidir; geçmişe bildirim planlanamaz.
Tolerans sabitleri testi	Birleştirme toleransları (120 sn / 50 km) belirlenen makul aralıkların dışına çıkarılmamalıdır.
Bozuk JSON testi	Alanları eksik, tipi yanlış veya null içeren JSON girdileri istisna fırlatmadan varsayılan değerlere düşmelidir.

12.4. Statik Analiz

Projede `flutter_lints` paketi etkin durumdadır ve kurallar `analysis_options.yaml` dosyasında tanımlanmıştır. Statik analiz; kullanılmayan değişkenler, eksik `const` tanımları, olası null hataları ve stil tutarsızlıkları gibi sorunları derleme öncesinde yakalar.

`kontrol.command` betiği, statik analizi ve tüm testleri tek komutta çalıştırarak derleme öncesi kontrolü tek adıma indirmiştir. Bu, tekrarlanan doğrulama adımlarının atlanmasını önlemiştir.

13. KARŞILAŞILAN SORUNLAR VE ÇÖZÜMLERİ

Bu bölüm, geliştirme boyunca karşılaşılan somut teknik sorunları ve uygulanan çözümleri belgelemektedir. Sorunlar kaynağına göre gruplanmıştır.

13.1. Veri Katmanı Sorunları

S1 — AFAD sayısal değerleri metin olarak gönderiyor

Belirti: `json['magnitude']` değeri doğrudan `double` tipine atandığında tip hatası oluşuyordu.

Sebeup: AFAD API'si tüm sayısal alanları string olarak serileştirmektedir (`"magnitude": "3.7"`).

Çözüm: `double.tryParse` ile güvenli dönüşüm; başarısızlık hâlinde varsayılan 0 değeri. İlgili durum `widget_test.dart` içinde test edilmiştir.

S2 — İki API'nin koordinat sırası farklı

Belirti: Kandilli kaynağı seçildiğinde harita işaretçileri Türkiye'nin çok dışında, anlamsız konumlarda beliriyordu.

Sebeup: AFAD ayrı `latitude` / `longitude` alanları verirken Kandilli GeoJSON standardına göre `[boylam, enlem]` dizisi göndermektedir.

Çözüm: Kandilli çözümleyicisinde dizinin 0. elemanı boylam, 1. elemanı enlem olarak okunmakta ve bu davranış hem kod yorumuyla hem testle korunmaktadır.

S3 — Türkçe karakterler bozuk görünüyordu

Belirti: “Çanakkale”, “İğdır” gibi yer adları bozuk karakterlerle görüntüleniyordu.

Sebeup: `http` paketinin `response.body` özelliği, sunucu açıkça karakter kümesi bildirmediğinde Latin-1 varsayabilmektedir.

Çözüm: Ham baytların açıkça UTF-8 olarak çözülmesi: `utf8.decode(yanıt.bodyBytes)`.

S4 — Tarih biçimleri uyuşmuyor

Belirti: Kandilli kayıtlarının tarihleri çözümlenemiyor, tümü “şimdi” olarak görünüyordu.

Sebeup: AFAD `2026-07-25T07:40:15`, Kandilli ise `2026-07-25 23:49:05` biçimi kullanmaktadır. `DateTime.tryParse` boşluklu biçimi kabul etmez.

Çözüm: Ayırıştırma öncesi ilk boşluğun `T` ile değiştirilmesi.

S5 — AFAD verisi gecikmeli yayınlanıyor

Belirti: “Son 24 saat” filtresi seçildiğinde liste bazen tamamen boş kalıyordu.

Sebeup: Ölçümde AFAD'ın en yeni kaydının yaklaşık 11,5–22 saat geriden geldiği tespit edilmiştir.

Çözüm: Üç katmanlı: (a) varsayılan kaynak **Otomatik** moda alındı ve iki kaynaktan taze olanı seçilir hâle getirildi; (b) varsayılan zaman aralığı 7 güne çıkarıldı; (c) kaynak gecikmeliyse arayüzde uyarı gösterilir.

S6 — Kandilli yalnızca son 24 saati veriyor

Belirti: Kandilli seçiliyken “Son 7 gün” filtresi sessizce etkisiz kalıyordu.

Sebeup: API tarih aralığı parametresi kabul etmemekte ve yanıtı 24 saatle sınırlamaktadır.

Çözüm: `DepremYanıt` sınıfına kapsama denetimi eklendi (`kapsamEksik`); arayüz kullanıcıya bu sınırı açıkça bildiriyor.

13.2. Performans ve Arayüz Sorunları

S7 — Kaydırıcı her harekette API isteği atıyordu

Belirti: Minimum büyüklük kaydırıcısı hareket ettirildiğinde saniyede onlarca HTTP isteği gönderiliyordu.

Sebeup: İstek `onChanged` olayına bağlanmıştı; bu olay parmak her kıpırdadığında tetiklenir.

Çözüm: `onChanged` yalnızca ekrandaki önizleme etiketini güncelliyor; asıl istek parmak kaldırıldığında tetiklenen `onChangeEnd` olayında atılıyor.

S8 — Sekme değişiminde harita sıfırlanıyordu

Belirti: Haritada bir bölgeye yaklaşıp listeye geçip geri dönüldüğünde harita başlangıç konumuna dönüyordu.

Sebe: Standart yapıda sekme değişiminde widget ağacı yeniden oluşturulmakta, durum kaybolmaktadır.

Çözüm: `IndexedStack` kullanılarak üç sekmenin de bellekte tutulması sağlandı.

S9 — Duyurular sekmesi çoğu zaman boş kalıyordu

Belirti: Test döneminde TRT manşet beslemesindeki 50 haberin hiçbiri deprem haberi değildi.

Sebe: Deprem haberi ancak kayda değer bir deprem olduğunda yayımlanır; o hafta en büyük deprem 4.1 büyüklüğündeydi.

Çözüm: Sekme iki katmanlı hâle getirildi. Resmî veriden üretilen bültenler kalıcı omurgayı oluştururken haberler zenginleştirme katmanı olarak konumlandırıldı.

13.3. Paket ve Platform Sorunları

S10 — `share_plus` paketi kullanılmadı

Belirti: Paylaşım özelliği için eklenen paket derlenmiyordu.

Sebe: Paketin güncel sürümü Flutter 3.38+ gerektiriyordu; ayrıca iPad'de `sharePositionOrigin` parametresi verilmediğinde çökme sorunu bilinmektedir.

Çözüm: Paylaşım, `Clipboard` API'si ile panoya kopyalama biçiminde yeniden tasarlandı. Ek paket gerekmiyor ve tüm platformlarda aynı biçimde çalışıyor.

S11 — Bildirim paketi API'si sürümler arasında değişiyor

Belirti: Paket güncellendiğinde derleme hataları oluşuyordu.

Sebe: `flutter_local_notifications` ana sürümlerde parametre yapısını değiştirmektedir.

Çözüm: Sürüm `^19.0.0` ile sabitlendi; caret 20.0.0'a geçmediği için API kararlılığı garanti altına alındı. Ayrıntı 8.5. bölümde.

S12 — Acil durum butonu sessizce çalışmıyordu

Belirti: Android 11 ve üzeri cihazlarda butona basıldığında hiçbir şey olmuyor, hata da görünmüyordu.

Sebe: Paket görünürlüğü kuralları gereği bildirilmemiş şemalar için `canLaunchUrl()` her zaman `false` döndürmektedir.

Çözüm: `sms`, `tel` ve `https` şemaları `AndroidManifest.xml`'deki mevcut `<queries>` bloğuna eklendi. Ayrıca tüm kanallar başarısız olursa mesajın panoya kopyalanacağı bir son yedek eklendi.

S13 — Açılışta beyaz parlama

Belirti: Uygulama her açıldığında koyu arayüzden önce beyaz bir ekran görünüyordu.

Sebe: İşletim sisteminin gösterdiği yerel açılış ekranı varsayılan olarak beyaz zeminlidir.

Çözüm: Android `launch_background.xml` ve iOS `LaunchScreen.storyboard` zeminleri logonun zemin rengiyle (`#0B131B`) eşitlendi.

S14 — iOS ikonunda beyaz kenar

Belirti: iOS'ta uygulama ikonunun köşelerinde beyaz bir çerçeve görünüyordu.

Sebe: Kaynak görsel yuvarlatılmış kare olarak tasarlanmış, köşe dışı beyaz bırakılmıştı; iOS kendi maskesini uyguladığı için bu alan görünür kalıyordu.

Çözüm: Köşelerden flood fill ile beyaz alan ikonun zemin rengiyle dolduruldu. İkon içindeki beyaz ev şekli dış bölgeye bağlı olmadığı için etkilenmedi.

S15 — İnternet izni eksikliği

Belirti: Uygulama Android'de açılıyor ancak “sunucu yanıt vermedi” hatası veriyordu.

Sebe: Android'de `INTERNET` izni açıkça bildirilmedikçe uygulama ağ erişimi yapamaz.

Çözüm: `AndroidManifest.xml` içine `<uses-permission android:name="android.permission.INTERNET"/>` satırı eklendi ve bu adım kurulum belgesine yazıldı.

14. BİLİNER SINIRLAR

Bir projenin sınırlarını bilmek ve açıkça belgelemek, o sınırların bulunmadığını iddia etmekten daha değerlidir. Aşağıda uygulamanın mevcut sürümündeki bilinen kısıtlar, nedenleriyle birlikte listelenmiştir.

Sınır	Neden	Kullanıcıya etkisi
Deprem sonrası anlık bildirim yok	Sürekli çalışan bir sunucunun API'yi izleyip itmeli bildirim (push) göndermesi gerekir	Yeni depremler yalnızca uygulama açıkken görülür
Erken uyarı yok	Sismik istasyon ağı veya işletim sistemi düzeyinde erişim gerektirir	Uygulama içinde resmî çözümlere yönlendirme yapılır
Acil mesaj otomatik gönderilmez	iOS ve Android platform kısıtı	Son “gönder” dokunuşu kullanıcıdadır; arayüzde açıkça belirtilir
Çevrimdışı çalışmıyor	Veriler ön belleğe alınmıyor	İnternet yoksa liste boş kalır
Şiddet tahmini yaklaşıktır	Zemin sınıfı (V_{s30}), bina türü ve kat yüksekliği hesaba katılmıyor; model M5.0 altında ekstrapolasyon yapıyor	Sonuç kaba kategori olarak ve “(tahmini)” etiketiyle sunulur
Artçı sayımı yaklaşıktır	100 km yarıçap ve “daha küçük büyüklük” kuralına dayanır; sismolojik bir artçı tanımı değildir	Etiket “3.0+ artçı: N” biçiminde eşik belirtilerek verilir
Saat dilimi doğrulanmadı	İki kurumun saatleri farklı dilimlerde (UTC / TSİ) olabilir	Gelen değer olduğu gibi gösterilir
Tek haber kaynağı	Anadolu Ajansı RSS'i sıkıştırılmış geliyor ve çözülemedi	Haber çeşitliliği sınırlı
Tahmin filtresi kelime tabanlı	Yeni ifade kalıplarını kaçırabilir	İddia içeren haber gizlenmez, işaretlenir
Kandilli API'si resmî değil	Üçüncü taraf servis üzerinden erişiliyor	Servis kapanırsa AFAD'a otomatik geçilir
Harita kümeleme yok	Kümeleme paketi entegre edilmedi	Yoğun bölgelerde işaretçiler üst üste binebilir
Yalnızca koyu tema	Uygulama koyu zemin üzerine tasarlandı	Açık tema seçeneği bulunmuyor
Sistem paylaşım menüsü yok	share_plus paketi sürüm uyumsuzluğu	Paylaşım panoya kopyalama ile yapılır

15. GELİŞTİRİLEBİLECEK YÖNLER

Aşağıdaki geliştirmeler, mevcut sınırları gidermeye ve uygulamanın değerini artırmaya yönelik olarak önceliklendirilmiştir.

Öncelik	Geliştirme	Gereksinim ve kazanım
Yüksek	Deprem sonrası itmeli bildirim	Kayıtlı yerlerde belirli bir şiddetin üzerinde tahmin edildiğinde bildirim gönderilmesi. Firebase Cloud Messaging ve API'yi sürekli izleyen bir sunucu gerektirir. Uygulamanın en büyük eksiğini kapatır.
Yüksek	Çevrimdışı ön bellek	Son çekilen verinin yerel veritabanına (SQLite/Hive) kaydedilip internet yokken gösterilmesi. Deprem sonrasında şebekenin çökebileceği düşünülürse kritik bir özellik.
Orta	Artçı özeti	“Son 24 saatte 47 artçı, en büyüğü 3.8, azalma eğiliminde” biçiminde derli toplu bir özet kartı.
Orta	Zemin sınıfı ile şiddet iyileştirmesi	V_{s30} verisi entegre edilerek tahminin yerel zemin koşullarına duyarlı hâle getirilmesi.
Orta	Harita işaretçi kümeleme	<code>flutter_map_marker_cluster</code> paketi ile yoğun bölgelerdeki üst üste binmenin giderilmesi.
Orta	Saat dilimi normalizasyonu	İki kaynaktan gelen zamanların tek bir dilime çevrilerek doğrulanması.
Düşük	Şehir bazlı istatistik ve grafikler	Belirli bir il için zaman içindeki deprem dağılımının grafikte gösterilmesi.
Düşük	Konum izniyle yakındaki depremler	Kullanıcının anlık konumuna göre otomatik filtreleme (isteğe bağlı özellik olarak).
Düşük	Açık tema seçeneği	Gündüz kullanımı ve kullanıcı tercihi için alternatif palet.
Düşük	Çoklu dil desteği	İngilizce arayüz ile Türkiye'de yaşayan yabancı kullanıcılara erişim.

16. ÇALIŞMA TAKVİMİ (30 İŞ GÜNÜ)

Staj süresi altı haftalık bloklar hâlinde planlanmış ve her haftaya belirli bir teslim hedefi atanmıştır. Aşağıdaki tablo, yapılan çalışmaların haftalık dağılımını göstermektedir.

Hafta / Gün	Ana Konu	Yapılan Çalışmalar
1. Hafta Gün 1–5	Araştırma, kurulum ve temel yapı	• Problem tanımı ve gereksinim analizi • Mevcut deprem uygulamalarının incelenmesi ve eksiklerin belirlenmesi • Flutter SDK, Android Studio ve Xcode kurulumu • AFAD ve Kandilli API'lerinin araştırılması, yanıt yapılarının incelenmesi • Proje iskeletinin oluşturulması, katmanlı klasör yapısının kurulması • Deprem modeli ve iki JSON çözümleyicisinin yazılması • İlk liste ekranının çalışır hâle getirilmesi
2. Hafta Gün 6–10	Veri katmanı ve harita	• DepremServisi geliştirilmesi, HTTP hata yönetimi ve zaman aşımı • UTF-8 kodlama sorununun çözülmesi (S3) • Koordinat sırası hatasının tespiti ve düzeltilmesi (S2) • flutter_map entegrasyonu, OpenStreetMap döşemeleri • Büyüklüğe göre renk ve boyut kodlu işaretçiler • DepremDeposu durum yönetimi sınıfının tasarımı • Alt sekme yapısı ve IndexedStack ile durum korunması (S8)
3. Hafta Gün 11–15	Filtreleme, tercihler ve şiddet modeli	• Hızlı filtreler ve ayrıntılı filtre paneli • Sunucu/cihaz filtre ayrımı ve kaydırıcı optimizasyonu (S7) • shared_preferences ile tercih hafızası • Şiddet sönümlenme modeli üzerine literatür araştırması • Allen ve ark. (2012) modelinin OpenQuake kaynağından doğrulanması • Haversine mesafesi ve MMI hesabının yazılması • "Yerlerim" özelliği, 81 il koordinat veritabanı, konum ekleme ekranları • siddet_hesabi_test.dart testlerinin yazılması
4. Hafta Gün 16–20	Hazırlık, bildirim ve acil durum	• Hazırlık kontrol listesi ve madde tanımları • flutter_local_notifications entegrasyonu ve sürüm sorunu (S11) • Yeniden planlama algoritmasının tasarımı ve uygulanması • Bildirim kimliği çakışma sorununun tespiti ve sabit kimliklere geçiş • Erken uyarı bilgilendirme ekranı • Acil durum modülü: basamaklı konum yedeklemesi, SMS/WhatsApp/112 • Android paket görünürlüğü sorununun çözümü (S12) • hatırlatma_test.dart ve acil_durum_test.dart testleri
5. Hafta Gün 21–25	Duyurular, kaynak stratejisi ve tasarım	• AFAD gecikmesinin ölçülmesi ve otomatik kaynak seçiminin geliştirilmesi (S5) • Kapsama şeffaflığı sınıfının eklenmesi (S6) • Bülten üreticisi: artçı sayımı ve eğilim hesabı • TRT Haber RSS entegrasyonu, XML ayrıştırma, RFC 822 tarih çözümü • Üç katmanlı haber süzgeci (deprem / Türkiye / tahmin iddiası) • Kayıt birleştirme mantığı ve birlestirme_test.dart • Koyu tema paleti, bilgi hiyerarşisi, iskelet yükleme, gün başlıkları
6. Hafta Gün 26–30	Kullanıcı deneyimi, kimlik ve teslim	• Kişisel durum kartının eklenmesi • İlk açılış tanıtım ekranlarının tasarımı • Erişilebilirlik çalışması: Semantics, kontrast, dokunma hedefleri, FittedBox • Uygulama ikon setlerinin üretilmesi, beyaz kenar sorununun giderilmesi (S14) • Animasyonlu açılış ekranı ve beyaz parlamanın giderilmesi (S13) • Dokunsal geri bildirim ve boş/hata ekranlarının iyileştirilmesi • Kalan testlerin tamamlanması (toplam 143 test) • Statik analiz düzeltmeleri, kod temizliği • Belgelendirme, README ve rapor hazırlığı, teslim

Çalışma Yönteminden Notlar

Süreç boyunca iki alışkanlık verimliliği belirgin biçimde artırmıştır. Birincisi, her tasarım kararının gerekçesini kod içine yorum olarak yazmaktır; bir hafta sonra aynı koda dönüldüğünde “neden böyle yapmıştım?” sorusuna cevap aramak zaman kaybettirmemiştir. İkincisi, bir hata bulunduğunda düzeltmeden önce o hatayı yakalayan testin yazılmasıdır; bu, aynı hatanın sonraki değişikliklerle geri gelmesini önlemiştir.

17. KAZANIMLAR VE DEĞERLENDİRME

17.1. Teknik Kazanımlar

Alan	Kazanım
Mobil geliştirme	Flutter çerçevesinin widget ağacı, yaşam döngüsü, bildirimsel arayüz yaklaşımı ve platform kanalları hakkında uygulamalı deneyim
Dart dili	Null güvenliği, <code>async / await</code> , <code>Future.wait</code> ile paralel işlem, <code>Stream</code> ve <code>Timer</code> kullanımı, fabrika kurucuları, enum'larda alan tanımlama
API entegrasyonu	REST servislerin tüketilmesi, farklı şemaların ortak modele indirgenmesi, hata ve zaman aşımı yönetimi, karakter kodlaması
Yazılım mimarisi	Katmanlı mimari, sorumlulukların ayrılması, bağımlılık yönünün kontrolü, tek yönlü veri akışı
Durum yönetimi	<code>ChangeNotifier</code> ve gözlemci deseninin (observer pattern) iç işleyişinin kavranması
Coğrafi hesaplama	Küresel koordinat sistemleri, haversine formülü, GeoJSON standardı, hiposantr mesafesi
Bilimsel model uygulaması	Hakemli bir yayındaki denklemin koda dökülmesi ve açık kaynak bir referans uygulamayla doğrulanması
Test	Birim testi yazımı, saf fonksiyon tasarımının test edilebilirliğe etkisi, sınır durum ve tutarlılık testleri
Platform bilgisi	Android izin modeli ve paket görünürlüğü, iOS ikon ve açılış ekranı kuralları, yerel bildirim altyapısının sınırları
Araç kullanımı	Git sürüm kontrolü, statik analiz, kabuk betikleri ile iş akışı otomasyonu

17.2. Mühendislik Yaklaşımına İlişkin Kazanımlar

Projenin teknik içeriğinin ötesinde, yazılım mühendisliğine bakış açısını değiştiren birkaç çıkarım elde edilmiştir.

Sessiz hatalar, görünür hatalardan tehlikelidir. Kandilli'nin 24 saatlik sınırı ve Android paket görünürlüğü sorunu; ikisi de hiçbir hata mesajı üretmeden yanlış sonuç doğuran durumlardı. Bir sistemin “çalışıyor gibi görünmesi”, çalıştığı anlamına gelmez. Bu deneyim, bir özellik yazıldıktan sonra “bu nasıl sessizce başarısız olabilir?” sorusunun sorulması alışkanlığını kazandırmıştır.

Kullanıcıya doğruyu söylemek bir tasarım kararıdır. Uygulamanın yapamadıklarını gizlemek kısa vadede daha etkileyici bir ürün üretti. Ancak bir afet uygulamasında kullanıcının yanlış varsayımla hareket etmesi gerçek zarar doğurur. Sınırları açıkça belirtmek ürünü zayıflatmamış, aksine güvenilir kılmıştır.

Görünmeyen bir özellik, olmayan bir özellikte aynıdır. Uygulamanın en değerli işlevi başlangıçta ayarların içinde gizliydi. Bu farkındalık, teknik geliştirme kadar keşfedilebilirliğin de tasarım sorumluluğu olduğunu göstermiştir.

Testin asıl değeri güven vermesidir. 143 test yazmanın kazancı yalnızca yakalanan hatalar değildir; asıl kazanç, bir bölümü yeniden yazarken bir şeyleri bozmuş olma kaygısı duymadan çalışabilmektir.

Gerekçeyi yazmak, kodu yazmak kadar önemlidir. Kaynak dosyalardaki yorumlar “ne yapıldığını” değil “neden böyle yapıldığını” anlatacak biçimde yazılmıştır. Bu, hem sonraki günlerde hızlı ilerlemeyi hem de bu raporun hazırlanmasını kolaylaştırmıştır.

17.3. Zorlanılan Noktalar

Sürecin en zorlayıcı bölümü, bilimsel bir yayındaki şiddet denkleminin doğru biçimde uygulanmasıydı. Yayındaki gösterim ile OpenQuake kaynak kodundaki uygulamayı karşılaştırmak, katsayıların hangi mesafe tanımına ait

olduğunu ayırt etmek ve modelin geçerlilik sınırlarını doğru yorumlamak beklenenden uzun sürmüştür. Ancak bu bölüm aynı zamanda en öğretici kısım olmuştur: bir alanın literatürünü okuyup ürüne dönüştürmenin nasıl bir süreç olduğu deneyimlenmiştir.

İkinci zorluk, iki veri kaynağının farklı davranışlarını tek bir tutarlı deneyime indirgemektir. Farklı şemalar, farklı gecikmeler, farklı kapsama süreleri ve farklı filtre yetenekleri; bunların hepsini kullanıcıya yansıtmadan yönetmek, mimarinin birkaç kez gözden geçirilmesini gerektirmiştir.

18. SONUÇ

Bu staj kapsamında, fikir aşamasından test edilmiş çalışan bir ürüne kadar tüm yazılım geliştirme yaşam döngüsü tek başına yürütülmüş ve **“Depremi Nabzi”** adlı çapraz platform mobil uygulaması ortaya çıkarılmıştır.

Proje, giriş bölümünde belirlenen yedi hedefin tamamını karşılamıştır:

Hedef	Tanım	Durum
H1	Kişiselleştirilmiş etki bilgisi	Karşılandı — Allen ve ark. (2012) modeli uygulandı, kullanıcı konumlarında kategori tabanlı hissedilirlik tahmini üretiliyor
H2	Afet öncesi işlevsellik	Karşılandı — hazırlık kontrol listesi ve yeniden planlamaya dayalı periyodik yerel bildirim sistemi çalışıyor
H3	Dürüst iletişim	Karşılandı — tahmin ve erken uyarı konusunda ayrı bilgilendirme ekranı; tüm tahminlerde geçerlilik etiketi
H4	Kesintisiz çalışma	Karşılandı — iki kaynağa paralel istek, tazeliğe göre otomatik seçim, tek kaynak hatasında sorunsuz devam
H5	Gizlilik	Karşılandı — konumlar yalnızca cihazda; takip için konum izni istenmiyor
H6	Maliyetsiz altyapı	Karşılandı — API anahtarı, faturalandırma ve arka uç sunucusu gerektirmiyor
H7	Erişilebilirlik	Karşılandı — ekran okuyucu etiketleri, renk+sayı çift kodlama, büyük yazı tipi uyumu, 48×48 dokunma hedefleri

Sayısal olarak proje; 39 kaynak dosyada 10.982 satır Dart kodu, 5 sekmeli ve 14 ekranlı bir arayüz, 8 servis sınıfı, 9 dış bağımlılık ve 33 grup içinde 143 birim testi içermektedir. Uygulama Android ve iOS platformlarında derlenip çalıştırılmış, her iki ortamda da test edilmiştir.

Teknik başarının ötesinde, projenin ayırt edici tarafı yaklaşımındadır. Piyasadaki deprem uygulamalarının büyük bölümü veriyi olduğu gibi aktarmakla yetinirken, bu proje kullanıcının gerçekte sorduğu soruyu — *“beni etkiler mi?”* — merkeze almış ve ham veriyi doğrudan kullanılabilir bir cevaba dönüştürmüştür. Aynı ölçüde önemli olan, uygulamanın yapamadıklarını da açıkça anlatmasıdır; bir afet uygulamasında kullanıcının yanlış varsayım ile hareket etmesinin bedeli gerçek olabilir.

Staj süreci, yazılım geliştirmenin yalnızca kod yazmaktan ibaret olmadığını uygulamalı olarak göstermiştir. Problem tanımlamak, kısıtlarla çalışmak, gerekçeli karar vermek, hataları belgelemek, sınırları dürüstçe iletmek ve sonucu anlaşılır biçimde aktarmak; bunların tamamı sürecin ayrılmaz parçalarıdır. Bu bakış açısı, stajın en kalıcı kazanımı olmuştur.

KAYNAKÇA

Bilimsel Yayınlar

1. Allen, T. I., Wald, D. J. & Worden, C. B. (2012). *Intensity attenuation in active crustal regions*. Journal of Seismology, 16(3), 409–433.
2. Wood, H. O. & Neumann, F. (1931). *Modified Mercalli Intensity Scale of 1931*. Bulletin of the Seismological Society of America, 21(4), 277–283.

Veri Kaynakları

1. T.C. İçişleri Bakanlığı Afet ve Acil Durum Yönetimi Başkanlığı (AFAD) — Deprem Katalog Servisi. <https://deprem.afad.gov.tr/apiv2/event/filter>
2. Boğaziçi Üniversitesi Kandilli Rasathanesi ve Deprem Araştırma Enstitüsü — Deprem kayıtları. <http://www.koeri.boun.edu.tr>
3. OpenStreetMap katkıda bulunanları — Harita döşemeleri. <https://www.openstreetmap.org>
4. TRT Haber — RSS haber beslemeleri. <https://www.trthaber.com>

Yazılım ve Teknik Belgeler

1. GEM Foundation — OpenQuake Engine, `AllenEtAl2012Rhypo` uygulaması. <https://github.com/gem/oq-engine>
2. Google — Flutter Belgeleri. <https://docs.flutter.dev>
3. Google — Dart Dil Turu. <https://dart.dev/language>
4. Google — Android Geliştirici Belgeleri: Paket görünürlüğü filtreleme. <https://developer.android.com/training/package-visibility>
5. Apple — Human Interface Guidelines: App Icons. <https://developer.apple.com/design/human-interface-guidelines>
6. United States Geological Survey (USGS) — Earthquake Prediction FAQ. <https://www.usgs.gov/faqs>
7. Flutter paket deposu (pub.dev) — `http`, `flutter_map`, `latlong2`, `shared_preferences`, `flutter_local_notifications`, `timezone`, `geolocator`, `url_launcher`, `xml` paket belgeleri. <https://pub.dev>

Kurumsal Hazırlık Kaynakları

1. AFAD — Deprem öncesi, sırası ve sonrasında yapılması gerekenler; deprem çantası içeriği.
2. Türk Kızılay — Afet hazırlığı önerileri.

Lisans ve kaynak bildirimi

Deprem verileri AFAD ve Boğaziçi Üniversitesi Kandilli Rasathanesi'ne aittir. Harita verileri © OpenStreetMap katkıda bulunanları. Haber içerikleri ilgili yayın kuruluşlarına aittir. Bu uygulama eğitim amaçlı bir staj projesidir ve resmî bir afet bilgilendirme aracı değildir.

EK A — PROJE DİZİN YAPISI

deprem_takip/		
lib/		
main.dart	40	Uygulama girişi, tema kaydı
models/		
deprem.dart	109	Veri modeli + iki JSON çözümleyici
kayitli_konum.dart	82	Kullanıcının takip ettiği yerler
hazirlilik_maddesi.dart	228	Hazırlık listesi ve durum modeli
acil_kisi.dart	163	Acil kişi modeli, telefon biçimleri
haber.dart	139	Haber modeli ve süzme kuralları
services/		
deprem_servisi.dart	487	AFAD/Kandilli istekleri, kaynak seçimi
tercih_servisi.dart	172	Cihaz depolaması sarmalayıcısı
bildirim_servisi.dart	257	Yerel bildirim sarmalayıcısı
konum_servisi.dart	198	GPS + basamaklı yedekleme
acil_durum_servisi.dart	175	Mesaj üretimi, SMS/WhatsApp/arama
haber_servisi.dart	192	RSS indirme ve ayrıştırma
bulten_uretici.dart	269	Artçı sayımı, bülten üretimi
hatirlatma_planlayici.dart	138	Sonraki hatırlatma tarihi hesabı
state/		
deprem_deposu.dart	698	Ortak durum (ChangeNotifier)
screens/		
ana_kabuk.dart	172	Alt sekme çubuğu, sekme yönetimi
liste_sekmesi.dart	651	Arama, hızlı filtreler, liste
harita_sekmesi.dart	327	Harita + seçim kartı
duyurular_sekmesi.dart	758	Haberler ve bültenler
acil_durum_ekrani.dart	775	Acil buton, kişiler, 112
ayarlar_sekmesi.dart	365	Kaynak seçimi, tercihler
detay_ekrani.dart	610	Detay + yerlerdeki etki + paylaşım
hazirlilik_ekrani.dart	551	Kontrol listesi + hatırlatmalar
erken_uyari_ekrani.dart	359	Erken uyarı bilgilendirmesi
tanitim_ekrani.dart	354	İlk açılış tanıtımı (3 ekran)
konum_ekle_ekrani.dart	339	Şehirden veya haritadan yer seçme
yerlerim_ekrani.dart	269	Kayıtlı yerlerin listesi
acilis_ekrani.dart	260	Animasyonlu giriş ekranı
acil_kisi_ekle_ekrani.dart	234	Kişi ekleme / düzenleme
widgets/		
deprem_karti.dart	276	Listedeki tek kart
durum_karti.dart	250	Listenin üstündeki kişisel özet
filtre_sayfasi.dart	199	Alttan açılan filtre paneli
iskelet_kart.dart	125	Yükleme sırasındaki taslaklar
durum_gorunumu.dart	101	Boş / hata ekranları
utils/		
siddet_hesabi.dart	217	Mesafe + tahmini sarsıntı şiddeti
tema.dart	159	Renk paleti ve bileşen stilleri
sehirler.dart	157	81 il koordinatı, konum simgeleri
gun_gruplama.dart	88	Listeyi günlere ayırma
buyukluk_stili.dart	39	Büyükliğe göre renk / etiket / boyut
test/		
acil_durum_test.dart	31	test
hatirlatma_test.dart	26	test
siddet_hesabi_test.dart	20	test
haber_test.dart	20	test
birlestirme_test.dart	17	test
widget_test.dart	15	test
gun_gruplama_test.dart	14	test
android/		Android platform dosyaları
ios/		iOS platform dosyaları
assets/logo.png		Kaynak ikon görseli (512×512)
pubspec.yaml		Bağımlılıklar ve varlık tanımları
analysis_options.yaml		Statik analiz kuralları
basla.command		Hızlı başlatma betiği
calistir-ios.command		iOS Simulator başlatma betiği
kontrol.command		Analiz + test betiği
README.md		Proje belgelendirmesi

TOPLAM: ~11.000 satır Dart / 143 test

EK B — ÖRNEK API YANITLARI

B.1. AFAD Yanıt Örneği

İstek:

```
GET https://deprem.afad.gov.tr/apiv2/event/filter
?start=2026-07-19T00:00:00&end=2026-07-26T23:59:59
&orderby=timedesc&limit=1000&minmag=3.5
```

Yanıt (düz dizi, kısaltılmış):

```
[
  {
    "eventID" : "638123456789",
    "location" : "Akçadağ (Malatya)",
    "latitude" : "38.3507",          <-- DİKKAT: metin
    "longitude" : "37.8618",         <-- DİKKAT: metin
    "depth" : "11.5",               <-- DİKKAT: metin
    "magnitude" : "4.1",            <-- DİKKAT: metin
    "type" : "ML",
    "date" : "2026-07-25T07:40:15",
    "country" : "Türkiye",
    "province" : "Malatya",
    "district" : "Akçadağ"
  },
  ...
]
```

B.2. Kandilli Yanıt Örneği

İstek:

```
GET https://api.orhanaydogdu.com.tr/deprem/kandilli/live?limit=500
```

Yanıt (sarmalanmış, kısaltılmış):

```
{
  "status": true,
  "metadata": {
    "date_starts": "2026-07-25 00:00:00",
    "date_ends" : "2026-07-26 00:00:00",  <-- yalnızca 24 saat
    "total": 187
  },
  "result": [
    {
      "earthquake_id": "20260725234905",
      "title" : "EGE DENİZİ",
      "date_time" : "2026-07-25 23:49:05",  <-- boşluk, "T" değil
      "mag" : 3.9,
      "depth" : 13.4,
      "geojson": {
        "type": "Point",
        "coordinates": [26.7123, 39.2456]  <-- [BOYLAM, ENLEM]
      },
      "location_properties": {
        "closestCity" : { "name": "Çanakkale", "distance": 42315 },
        "epiCenter" : { "name": null },
        "closestCities" : [ ... ],
        "airports" : [ ... ]
      }
    },
    ...
  ]
}
```

B.3. Ortak Modele Dönüşüm

Her iki yanıt da aşağıdaki ortak modele indirgenir. Uygulamanın geri kalanı hangi kaynaktan geldiğini bilmek zorunda değildir.

```
class Deprem {  
    final String id;  
    final String yer;  
    final double buyukluk; // Richter / lokal büyüklük  
    final double derinlik; // km  
    final double enlem;  
    final double boylam;  
    final DateTime tarih;  
    final String kaynak; // "AFAD" veya "Kandilli"  
}
```

EK C — KURULUM VE ÇALIŞTIRMA

C.1. Gereksinimler

Bileşen	Not
Flutter SDK 3.27 veya üzeri	Dart 3.6+ içerir
Android Studio	Android SDK, emülatör ve platform araçları için
Xcode (yalnızca macOS)	iOS Simulator ve iOS derlemesi için
Yaklaşık 10 GB boş disk alanı	SDK'lar ve emülatör imajları
İnternet bağlantısı	Paket indirme ve uygulama çalışma zamanı

C.2. Adımlar

1. Platform klasörlerinin oluşturulması (depoda yalnızca `lib/` ve `pubspec.yaml` bulunuyorsa):

```
cd deprem_takip

flutter create --project-name deprem_takip \
  --org com.tunakilic \
  --platforms=ios,android .
```

Bu komut `lib/` klasörüne ve `pubspec.yaml` dosyasına dokunmaz; yalnızca eksik platform klasörlerini üretir.

2. Bağımlılıkların indirilmesi:

```
flutter pub get
```

3. Android internet izninin eklenmesi. `android/app/src/main/AndroidManifest.xml` dosyasında `<application>` satırının *üstüne*:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

En sık atlanan adım

Bu izin eklenmezse uygulama açılır ancak veri çekemez ve “sunucu yanıt vermedi” hatası verir. Hata mesajı ağ sorunuymuş gibi görüldüğü için kaynağı bulmak zaman alır.

4. Acil durum modülü için şema bildirimi. Aynı dosyadaki mevcut `<queries>` bloğunun içine `sms`, `tel` ve `https` şemaları eklenmelidir (9.5. bölümdeki örnek).

5. Çalıştırma:

```
# Bağlı cihazları listele
flutter devices

# Uygulamayı başlat
flutter run

# Alternatif: hazır betikler
./basla.command # varsayılan cihazda başlat
./calistir-ios.command # iOS Simulator'da başlat
```

C.3. Test ve Analiz

```
# Tüm birim testlerini çalıştır (143 test)
flutter test

# Statik analiz
flutter analyze

# İkisini birden
./kontrol.command
```

C.4. Yayın Derlemesi

```
# Android APK
flutter build apk --release

# Android App Bundle (Play Store için)
flutter build appbundle --release

# iOS (imzalama yapılandırması gerektirir)
flutter build ios --release
```

Rapor Sonu

Bu rapor, 30 iş günlük yaz stajı kapsamında geliştirilen “Depremin Nabzı” mobil uygulamasını belgelemektedir. Proje kaynak kodu, ayrıntılı teknik belgelendirme ve kurulum yönergeleri proje deposundaki README.md dosyasında yer almaktadır.